PROJECT HANDBOOK · DESIGN & TECHNICAL REFERENCE

TeachTrack
Complete documentation.

A detailed guide to setup, architecture, roles, workflows, data, forms, routes, configuration, security, operations and the complete colour system.

Prepared for Zaman & Saloni
Source review: 21 October 2026
Documentation describes the code in this workspace; it does not certify a live cloud deployment.

TeachTrack project handbook

Documentation edition: 21 October 2026. Prepared for Zaman & Saloni. This handbook describes the implementation in this workspace, including the updated role sign-in pages, staff account forms, public header/footer, seven-photo homepage carousel and Campus Life page.

The project is a server-rendered Django academic workspace. It has one shared campus workspace and three application roles: Student, Faculty and Director. Its source retains the original ClassApp/Classroom layout, academic relationships and migration history while using Django authentication for current accounts.

Contents

1. Project overview
2. Architecture and request lifecycle
3. Repository and source responsibilities
4. Installation and local startup
5. Accounts, roles and permissions
6. Page-by-page user guide
7. Academic workflow rules
8. Database and migration design
9. Forms and validation
10. URLs and request contracts
11. Design, colours and front-end behavior
12. Authentication, security and private files
13. Configuration and dependencies
14. Management commands
15. Testing and acceptance
16. Deployment, maintenance and recovery
17. Troubleshooting
18. Scope, limitations and extension guide
19. Glossary

The accompanying references provide every project model field, form field, route and explicit active-stylesheet colour: Database reference, Forms reference, Route reference, Configuration reference, Design system, Colour index and Test catalogue.

1. Project overview

Purpose

TeachTrack provides a central place for learning resources, quizzes, announcements, student questions, faculty feedback and complaint tracking. It replaces scattered academic exchanges with authenticated workflows whose permissions are checked by the server.

Students use the platform to find materials, attempt quizzes, ask doubts, send feedback and follow concerns. Faculty publish materials and quizzes, answer their assigned doubts and respond to assigned complaints. Directors manage campus access, review aggregate information, manage complaint status and export reports.

Functional scope

Module	Main capability	Principal records
Public website	Homepage, campus information and contact form	Static images; Contact
Identity	Username/password login, role labels, signed-in password change	Django User, Profile, LoginThrottle
Accounts	Director-created student/faculty/director accounts; deactivation	User, Profile and legacy role records
Resources	Notes, PYQs and assignments; search, filters and downloads	Resource, Download
Quizzes	Timed availability, question builder, exact-set scoring and review	Quiz, Question, Option, Submission
Announcements	Role-targeted campus posts and optional documents	Announcement
Notifications	Individual inbox, broadcasts and workflow messages	Notification
Doubts	Student-to-faculty questions and answers	Doubt
Feedback	One editable student rating/comment per faculty member	Rating
Complaints	Assigned concern, conversation, status and timeline	Complaint, ComplaintEvent
Reporting	Staff charts and director CSV exports	Queries over academic records

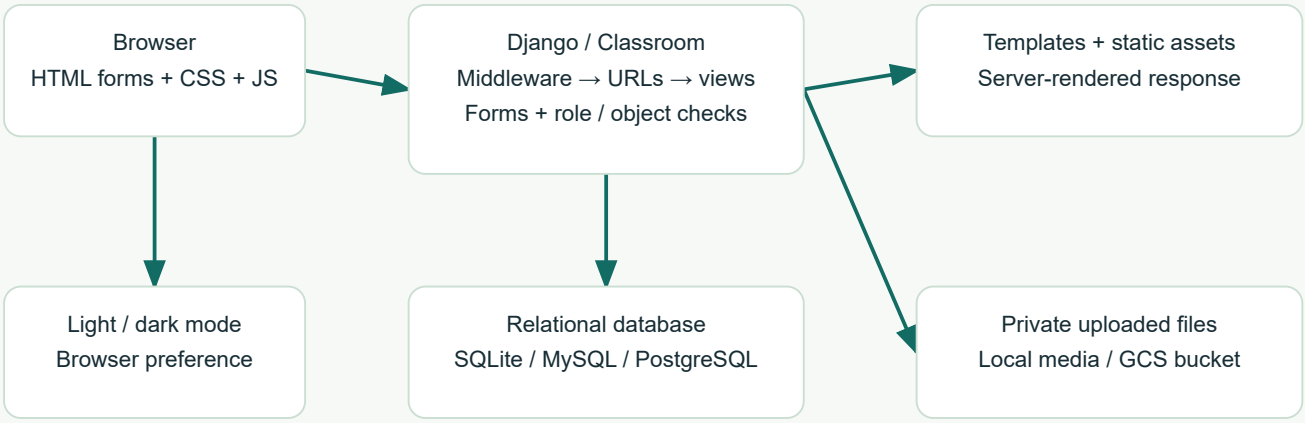
Implementation snapshot

Area	Current implementation
Framework	Django 5.2.18, as pinned in requirements.txt
Rendering	Django templates and ordinary HTML form submissions
Browser code	Project CSS and vanilla JavaScript; no front-end build step
Data access	Django ORM; database selected by DATABASE_URL
Local fallback database	db.sqlite3 only when DATABASE_URL is blank
Database options	SQLite for local work; PostgreSQL or MySQL Connector
Uploaded files	Local media in development; private GCS or disabled uploads in production
Static assets	Django staticfiles plus WhiteNoise compressed manifest storage
Production process	Vercel Django deployment or Gunicorn serving Classroom.wsgi:application in the container
Timezone	Asia/Kolkata; timezone-aware application timestamps
Tests	57 test methods in the current source catalogue

The source contains cloud configuration and a deployment guide. A working local server and passing local tests do not establish that a live deployment or private production bucket has been provisioned.

2. Architecture and request lifecycle

TeachTrack application architecture



Application layers

Layer	Responsibility	Source
Browser	Displays pages; submits CSRF-protected forms; runs menu/carousel/question-builder controls	Templates; app.css; app.js
Project configuration	Loads environment settings, middleware, installed apps and root routes	Classroom/settings.py; Classroom/urls.py
Routing	Maps public, academic and compatibility paths to view functions	ClassApp/urls.py
Authorization	Requires login, verifies roles and restricts object querysets	permissions.py; view checks
Form validation	Checks field values, account details, choices, file content and quiz structure	forms.py
Workflow logic	Coordinates queries, transactions, grading, notifications and responses	views.py
Shared services	Creates notifications and links current users with legacy records	services.py
Persistence	Describes records, constraints, indexes and relationships	models.py; migrations/
Presentation context	Supplies role and unread-notification count to authenticated templates	context_processors.py

Example: uploading a resource

1. An authenticated faculty member opens the upload form.
2. The role decorator admits faculty and directors; students cannot access the action by entering its URL directly.
3. The browser submits the form and file with a CSRF token.

4. Request-size middleware rejects a declared body above 12 MiB. Form validation checks the file itself against the 10 MiB policy and its permitted content format.
5. The view starts a database transaction, assigns the current user as uploader, strips directory components from the original filename and saves the resource.
6. Django storage writes the file under a generated resource key. Student recipients receive individual notification records.
7. A success message is added and the browser is redirected to the resource list.

Database transactions protect related database changes. They do not create a distributed transaction spanning SQL and an external object-storage write; an interrupted workflow can require orphan-file review.

Middleware sequence

The configured order is SecurityMiddleware, WhiteNoise, SessionMiddleware, CommonMiddleware, RequestSizeLimitMiddleware, CsrfViewMiddleware, AuthenticationMiddleware, MessageMiddleware and XFrameOptionsMiddleware. Ordering is significant: declared oversized bodies are checked before CSRF processing, and the custom middleware can add private/no-store headers after authenticated responses return.

There is no task queue, websocket consumer, public JSON API, service worker or separate JavaScript application in the current implementation. Notifications are created synchronously and their unread count refreshes on page navigation.

3. Repository and source responsibilities

```
TeachTrack/
  manage.py                         Django command entry point
  requirements.txt                  Main pinned dependency set
  requirements-mysql.txt            Compatibility include of the main requirements
  Dockerfile                        Python 3.12 / Gunicorn container
  .env.example                      Safe configuration template
  Classroom/
    settings.py                     Environment, database, storage and security
    urls.py                         Admin and account/password routes
    wsgi.py                         WSGI application entry point
    asgi.py                         ASGI application entry point
  ClassApp/
    models.py                       24 project models
    forms.py                        User-input and upload validation
    views.py                        Academic and public workflows
    urls.py                         Primary paths and original aliases
    permissions.py                  Login + allowed-role decorator
    services.py                     Notification and legacy-link helpers
    signals.py                      Profile creation when a User is created
    context_processors.py           Role and unread inbox count
    middleware.py                   Body-size and authenticated-cache controls
    admin.py                        Trusted Django admin registration
    apps.py                         App registration and signal loading
    tests.py                        Workspace and migration test classes
    management/commands/            Director, demo, report and import commands
    migrations/                     Original 0001–0006 and upgrade 0007–0009
    Templates/                      Active and retained legacy templates
    static/ClassApp/app.css         Current visual system
    static/ClassApp/app.js          Current browser interactions
    static/img/                     Campus photographs and other source images
    static/css/, js/, fonts/        Retained original theme assets
  docs/                             Handbook, references and operational guides
  tools/build_documentation.py      Regenerates references and offline HTML
  media/                            Local uploaded files, when local storage is used
```

```
staticfiles/                    Generated collectstatic output
.venv/                          Local Python environment
```

Source files versus generated/local files

Edit source images and styles under ClassApp/static/. Run collectstatic to produce staticfiles/ and its hash-named copies. Editing a generated file such as staticfiles/img/Slider-img1.<hash>.jpg is not a durable source change.

The environment file, generated development secret, local database, uploaded media, generated static output and logs are excluded by the project's ignore rules. Documentation contains example configuration values, not the contents of the private .env or stored account credentials.

Active template families

Family	Templates and behavior
Application shell	base.html; authenticated sidebar/topbar/footer, or anonymous authentication layout
Public shell	public_base.html overrides the page_layout block; shared public header/footer
Public pages	index.html; campus_life.html
Current sign-in	registration/login.html with the optional role label and role links
Generic forms	form.html for registration, uploads, contact and several action forms
Account administration	accounts.html and account_form.html
Overview and resources	dashboard.html and resources.html
Quizzes	quizzes.html; quiz_create.html; quiz_attempt.html; quiz_result.html; leaderboard.html
Communication	announcements.html; notifications.html; doubts.html; feedback.html
Complaints	complaints.html and complaint_detail.html
Reporting/history	reports.html and submission_history.html
Shared partials	pagination.html; signed-in password-change completion template
Error presentation	403.html, 404.html, 500.html; actual error presentation also depends on DEBUG

Files such as faculty_login.html, director_login.html, student_dashboard.html, Director_dashboard.html and the old quiz/ templates remain in the repository for historical context. Their presence does not mean a current URL renders them. The original role URLs now use the upgraded views and current templates. For example, /faculty-login/ renders registration/login.html with a faculty role context, and /faculty_dashboard/ renders dashboard.html after its role check.

Shared helper functions

Helper	Purpose
paginated	Creates a 15-record Paginator page and tolerates invalid page selection through get_page
form_page	Renders a form using form.html and a supplied page title
display_name	Uses full name when available, otherwise username
visible_complaints	Builds the student-own, faculty-assigned or director-all complaint queryset

Helper	Purpose
valid_quiz	Requires questions, at least two options each, at least one correct option, and exactly one correct option for single-selection questions
csv_value	Converts values to text and prefixes suspicious spreadsheet-formula starts with an apostrophe
notify	Deduplicates recipient IDs and bulk-creates notifications in batches of 500
audience_users	Selects active users, optionally restricted to one application role
ensure_legacy	Ensures the role record and original complaint wrappers exist for a current User
create_profile signal	Creates a Profile for each new User; superusers start as directors and other users as students

Trusted administration

admin.py registers 22 project models with Django's admin site, alongside Django's own User/Group administration. ProfileAdmin provides role/department filters and username/name/enrollment search. StudentAdmin displays academic fields, offers name/enrollment/email search and makes the User link read-only. FacultyAdmin and DirectorAdmin display contact/identity information and also make the User link read-only. The remaining registered project models use the default admin interface.

LoginThrottle and the historical PasswordResetOTP model are not registered by this admin.py. The site header is TeachTrack administration and its browser title is TeachTrack. This trusted interface can expose more underlying record data than the role-filtered product screens; its Django staff/permission requirements must be managed separately from Profile.role.

4. Installation and local startup

Choose the correct starting point

Situation	Correct action
This existing working workspace	Keep the current .env and virtual environment; use the short startup commands below
Fresh source checkout with no data	Create a virtual environment and .env, select a new database, then migrate
Existing consistent populated database	Read UPGRADE.md and rehearse against a backup before applying migrations
Historical incomplete bundled SQLite file	Do not fake migrations; use a clean database or the supported read-only importer after review

The Dockerfile's baseline is Python 3.12. The existing Windows workspace has also been exercised with Python 3.14.5 and Django 5.2.18. Keep interpreter and installed packages together; the direct .venv interpreter path avoids PowerShell activation-policy problems.

Start this existing Windows workspace

Run from the directory containing manage.py:

```
$env:DEBUG = 'True'
.\.venv\Scripts\python.exe manage.py check
.\.venv\Scripts\python.exe manage.py runserver
```

Open http://127.0.0.1:8000/. The DEBUG override matters if an IDE or parent process supplies a different DEBUG value: python-dotenv loads .env without overriding an already-set process environment variable.

Fresh Windows setup

Use an installed Python matching the chosen baseline. These commands assume Python 3.12 is installed and the source directory does not already contain a working environment:

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
if (-not (Test-Path -LiteralPath '.env')) { Copy-Item .env.example .env }
$env:DEBUG = 'True'
.\.venv\Scripts\python.exe manage.py check
.\.venv\Scripts\python.exe manage.py migrate
.\.venv\Scripts\python.exe manage.py create_director --username director --email director@example.com --name "Campus Director"
.\.venv\Scripts\python.exe manage.py runserver
```

Before migrate, confirm DATABASE_URL selects the intended new database. If it is blank, Django uses the project's db.sqlite3 path; an existing file at that path is not automatically replaced or cleaned.

Linux/macOS equivalent

```
python3.12 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
test -f .env || cp .env.example .env
export DEBUG=True
.venv/bin/python manage.py check
.venv/bin/python manage.py migrate
.venv/bin/python manage.py create_director --username director --email director@example.com --name "Campus Director"
.venv/bin/python manage.py runserver
```

Database choices

DATABASE_URL is parsed by dj-database-url. Local SQLite is the fallback when blank; production requires PostgreSQL or MySQL. The main dependency set includes psycopg and mysql-connector-python. requirements-mysql.txt includes the same base set for compatibility.

For MySQL, settings remap django.db.backends.mysql to mysql.connector.django and enable certificate and hostname verification with the public certs/aiven-ca.pem. Use the actual Aiven host, port, database name and URL-encoded credentials; keep credentials private. Schema migrations do not transfer data between engines.

First account and first useful session

The create_director command prompts privately for the password twice. There is no bundled default password. Sign in with the username chosen for that account. Use Manage accounts to create a faculty user, then add a student through public registration or the director account form. Uploaded resources and faculty choices become meaningful once those accounts exist.

Running createsuperuser is a separate decision for trusted Django administration. A normal director account is an application account and is not automatically a Django staff or superuser account.

5. Accounts, roles and permissions

Identity model

Django User stores the username, password hash, name, email and is_active status. Profile stores the application role. Original Student, Faculty and Director records retain descriptive/contact data and connect to the Django user. Faculty and student complaint workflows also use the original Teacher and AuthenticatedStudent wrappers.

Passwords are checked through Django AuthenticationForm. The older role-table username/password approach and custom session keys do not establish current identity. Plaintext pw columns were removed by migration 0009.

Own profile editing

Every role opens /accounts/profile/ from the sidebar name/role footer. Name, username, email, department and phone are editable for the current account; students additionally edit enrollment, year, semester, section and batch. Saving synchronizes linked role records and complaint-wrapper names. Role, account IDs and permission flags cannot be edited here. Duplicate usernames/emails fail validation.

Role-specific login

Page	Behavior
/accounts/login/	Shared username/password sign-in; workspace follows the account's role
/student-login/	Student-labeled sign-in; a valid account with another role receives guidance to use its own role page
/faculty-login/	Faculty-labeled sign-in; explains that a director creates faculty accounts
/director-login/	Director-labeled sign-in; explains existing-director account administration

Choosing a role link changes the sign-in context; it does not grant that role. Correct credentials on an incorrect role page do not count as an invalid-password attempt. An already-authenticated user who opens a login page is redirected to their dashboard.

Account-form differences

Input	Student	Faculty	Director
Username, full name and email	Required	Required	Required
Password and confirmation	Required	Required	Required
Course/department	Required; Course / department label	Required; Department label	Required; Department label
Enrollment	Required	Absent	Absent
Phone	Optional, maximum 10 characters	Optional, maximum 15 characters	Optional, maximum 15 characters
Year, semester and section	Optional	Absent	Absent
Who can create the account	Public student registration or a director	Director	Bootstrap command or existing director

Manage accounts → Create account defaults to Faculty. Role links select the form before data entry. The original faculty-registration and director-registration routes fix their intended role on the server, even if a submitted role value is different. Staff creation discards student-only form fields instead of saving inappropriate enrollment/year/semester data.

Permission matrix

Action	Student	Faculty	Director
Public homepage and campus information	Yes	Yes	Yes
Search/download resources	Yes	Yes	Yes
Upload a resource	No	Yes	Yes
Create/publish a quiz	No	Own quizzes	All quizzes
Attempt a quiz	Own single attempt	No	No
Review submissions/history	Own	Quizzes created by the faculty user	All
View leaderboard	Published quiz or previously attempted unpublished quiz	Own quizzes	All
Create a doubt	Yes	No	No
View doubts	Own	Assigned	All
Answer a doubt	No	Assigned only	No answer action
Submit/edit feedback	Own	No	No
View feedback comments	Own	Received	All
Post announcements or broadcasts	No	Students audience only	Any configured audience
Create a complaint	Yes	No	No
View/reply to complaints	Own	Assigned	All
Select complaint status	No	First pending reply starts progress automatically	Yes
Create/deactivate users	No	No	Yes, with director/superuser protections
Export reports	No	No	Yes
Use Django admin	Only if separately granted Django admin permissions	Same	Same

Feedback templates hide student identity from the displayed comments, but the underlying records retain student names/email/user links. This is presentation-level identity hiding, not irreversible anonymization of the stored data.

Deactivation and role maintenance

The director’s toggle action changes User.is_active. It protects the requesting user, director accounts and superusers from deactivation through that page. Inactive users cannot log in through the standard backend, and existing sessions no longer authenticate as an active user on subsequent requests.

There is no user-facing role-editing or profile-editing workflow in this version. A trusted administrator who modifies roles directly must keep the legacy role records and complaint wrappers consistent; changing one Profile field alone is not a complete role-migration procedure.

6. Page-by-page user guide

Public homepage

The header offers Home, Features, Campus life, Contact and account access. Signed-in users see My dashboard. The main section introduces TeachTrack and offers a workspace/sign-in action. Seven campus photos appear in a manual carousel with previous/next arrows, individual dots and a current-photo counter. The removed star/tagline block is absent.

The three feature cards introduce the resource library, quizzes/feedback and campus communication. The footer contains campus identity, public navigation, role-specific sign-in links, student registration, credits and Back to top.

Campus Life

Campus life opens /campus-life/, rather than scrolling to the homepage photograph. The page displays the user-selected campus cover image, SAIT logo, library and Common Ground café, four facility summaries, an eight-photo gallery, campus location and official contact links. With the cover/logo, facility images and location image, the page contains 13 image elements.

Campus descriptions link to the official institute pages. Image references and information sources are recorded in CAMPUS_CONTENT.md. Google thumbnail references were downloaded into project static assets so local rendering does not depend on third-party image requests.

Dashboard

Student cards count available quizzes, the student’s submissions, all resources and their open complaints. Faculty cards count their quizzes, submissions to their quizzes, their resource uploads and assigned open complaints. Director cards count active students, active faculty, all resources and all open complaints.

Recent resources and audience-relevant announcements are shown to all signed-in users. Staff receive participation and feedback charts; students do not see those staff chart panels. Complaint status totals use each role’s visible complaint queryset. The old Teacher Performance and Student Performance sections are not present.

Resources

Use search, category, subject and sorting controls to locate a file. Upload buttons are shown to staff and are also protected by server checks. A Download form sends POST and returns an attachment response. Download counts increase when that authenticated response is initiated.

Quizzes and submission history

Students see published quizzes, including those that may be upcoming or already closed; the attempt action performs the opening/closing check. Faculty see their own quizzes; directors see all. A six-character code filters the listing by an exact code. A student with an existing attempt is sent to the result rather than receiving another attempt.

Submission history shows only the student’s records, records for a faculty member’s own quizzes, or all records for a director. The leaderboard orders verified-student attempts by descending score, earlier submission time and ID; it uses sequential row positions rather than shared numerical ranks for equal scores.

Announcements and notifications

Announcements are shared posts filtered by audience, with an author exception that permits seeing one’s own posts. Notifications are separate recipient-specific inbox records. Marking one read can navigate to a validated local link; marking all read affects only the current user’s unread items.

Doubts

A student chooses an active faculty member, supplies a question and may add a validated attachment. The assigned faculty member can answer or update an answer. A director can inspect the records but does not receive the faculty answer action.

Faculty feedback

A student selects active faculty, chooses a rating from 1 to 5 and may add a comment. Submitting again updates the existing student/faculty feedback pair. The overview shows averages and response counts; faculty comments and aggregates are restricted to the receiving faculty user, while directors see the broader set.

Complaints

A student chooses the responsible faculty member and enters a title and concern. The detail view displays the conversation in chronological order. Students and assigned faculty can reply; directors additionally receive a status selector. The list can be filtered by the stored status code.

Reports and account administration

Directors can create accounts, deactivate eligible non-director users and download four report families. Accounts are listed by first name then username. Reports summarize data already in the system; they do not perform cohort enrollment, payroll, attendance or gradebook management.

Contact

The public Contact page uses public_base.html, retaining the homepage header, footer, active navigation and theme toggle. Valid submissions create Contact records and redirect home with a success message. Authorized Django administrators can inspect them at /admin/; they do not appear in the role dashboards and do not send email.

7. Academic workflow rules

Resource upload and download

Resources have a title, optional description, subject, category, file, safe original filename, uploader, count and timestamp. Valid categories are notes, pyq and assignment. New resource uploads notify active students. Uploaded resource keys use a random UUID and the lowercased file extension under resources/.

Search matches title, description or subject. Category filters use the stored code; subject uses a case-insensitive exact match. The title sort orders title then ID; the default is newest creation time then ID. Lists use 15 records per page.

The download view first opens the file. A missing file returns 404 before incrementing its count. Successful initiation creates a Download row and increments Resource.download_count with a database F expression. Repeat downloads count again; the system does not measure whether the client completed the entire transfer.

Building a quiz

1. Faculty or director enters title, subject, launch time and submission deadline, displayed in IST.
2. New QuizForm instances publish automatically and require name, enrollment, email and course. The five old checkbox settings are not displayed; authorized staff can still use the publication toggle later.
3. Add between 1 and 50 questions, with optional images and single/multiple selection.

4. Options A and B are required; C and D are optional. Mark exactly one correct option for single-choice, or a valid nonempty set for multiple-choice.
5. A transaction saves the quiz/questions/options and notifies active students.

Quiz question numbers sit inside each card above the question text. Question panels align with student details and long options wrap. The multipart form supports optional images when uploads are enabled. Quiz codes use secrets.token_hex(3).upper(); a uniqueness failure is reported rather than silently retried.

Availability and attempt eligibility

The eligibility condition is published and starts_at <= now < expiry_time. Equality with the opening time permits an attempt; equality with the closing time does not. An unpublished quiz cannot be newly attempted. A quiz also needs at least one valid question with at least two options per question and an appropriate correct-answer set.

The date widgets use local datetime input labels in IST. Django stores timezone-aware values and renders them using Asia/Kolkata. There is no separate browser countdown or per-student duration setting.

Attempt processing and scoring

The view locks the Quiz row inside a transaction, checks for an existing authenticated attempt, verifies availability and validates the quiz structure. Initial student details come from the authenticated account and linked student record. Course, year, semester, section and batch are separate prefilled inputs; submissions retain these values in academic_details JSON. The legacy class_div summary remains for compatibility. Empty optional values are filled from those initial details when saving.

Every question must have a non-empty answer selection. Selected IDs must belong to that question, may not repeat and must have exactly one element for single-selection questions. Forged option IDs from another question are rejected.

```
correct_for_question = sorted(all correct Option IDs)
selected_for_question = sorted(submitted Option IDs)
question_score = 1 if selected_for_question == correct_for_question else 0
submission_score = sum(question_score for every question)
submission_total = number of questions
percentage = round(score * 100 / total), or 0 when total is 0
```

Multiple-selection questions use exact-set scoring, with no partial marks. Selecting one correct option while omitting another is incorrect; selecting an extra incorrect option is also incorrect. There is no negative marking or configurable question weighting.

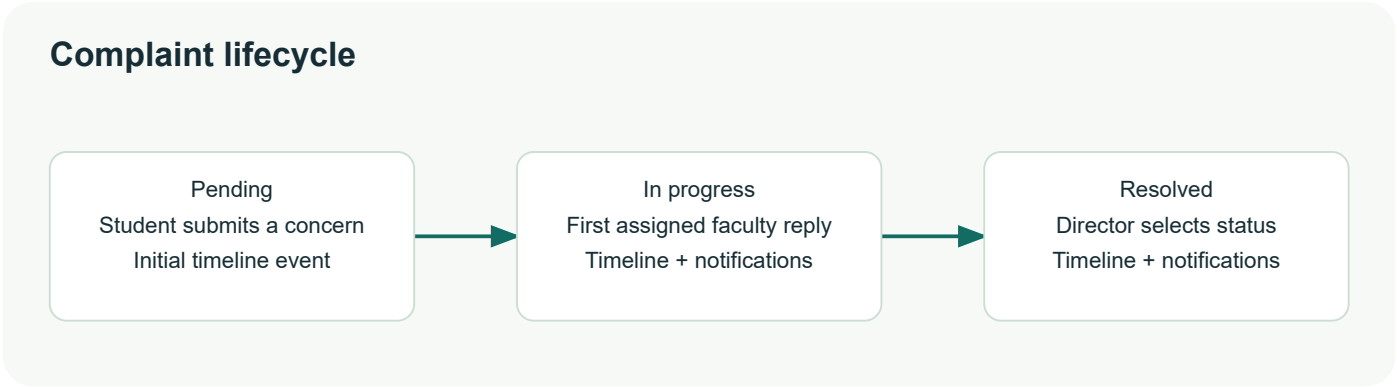
Saving an attempt writes Submission, a grading review JSON snapshot, a SubmissionAnswer row for each question and a SubmissionDetail row for each question. Both original answer models retain their selected-option M2M links. A database uniqueness constraint protects the quiz/student pair, supplementing the transaction check. A duplicate write is treated as an already-recorded submission.

Results, review and leaderboard

Students see their own saved score immediately. The template releases selected/correct answer review to students only once the quiz closes. Staff who are authorized for that quiz can review its answers earlier. Result views do not publicly expose the grading snapshot as a JSON endpoint.

A faculty member cannot inspect another faculty creator’s results. Directors can inspect all. Student leaderboards omit historical rows without a verified student link. A published quiz’s leaderboard is accessible to students; an unpublished quiz also requires that the student previously attempted it. These checks do not make the quiz code a secret authorization token.

Complaint lifecycle



Event	Status behavior	Records/recipients
Student creates concern	pending	Complaint plus initial ComplaintEvent; assigned faculty and active directors are notified
Assigned faculty first replies while pending	in_progress	Latest teacher_reply/reply_time plus a new event; participants/directors except actor are notified
Student replies	Preserves current status	Adds conversation event and notifications
Faculty replies after progress/resolution	Preserves current status	Updates latest faculty reply and adds event
Director replies/selects a status	Uses selected status, or current value when blank	Adds event and participant notifications

The UI's intended sequence is pending → in_progress → resolved. The director selector accepts any configured status, so reopening or moving back is technically allowed; the implementation does not enforce a one-way state machine. Directors must provide a reply message when submitting the form, even when their purpose is to change status.

Doubts and feedback

A doubt stores the authenticated student link and name/email snapshots, selected Faculty, subject, text, optional attachment and latest answer/timestamp. The assigned faculty user receives the new-doubt notification. An answer notifies the verified student when that link exists.

Feedback uses update_or_create on student/faculty. Uniqueness and the 1–5 range are enforced by database constraints as well as the form. Student identity hiding applies to the rendered feedback view. Historical duplicate/unlinked rows may remain in the database and affect staff aggregates.

Broadcasts and event notifications

Faculty can send announcements and direct inbox broadcasts to Students. Directors may target Everyone, Students, Faculty or Directors. Audience recipients are active users. Announcement content creates a notification whose message is truncated to the first 250 characters; a direct broadcast uses its entered message.

Workflow events include student welcome, new resources, quiz creation/publication/submission, new doubts/answers and complaint creation/updates. The notify helper deduplicates recipients before bulk insertion. Notifications do not send email, browser push or SMS.

Analytics and CSV exports

Report kind	Columns	Order
feedback	Faculty, Average rating, Responses	Faculty name, ID

Report kind	Columns	Order
quizzes	Quiz, Code, Faculty, Submissions, Active students, Participation %	Quiz title, ID
complaints	Title, Student, Faculty, Status, Created at	Complaint title, ID
resources	Title, Subject, Category, Downloads, Uploader	Resource title, ID

The CSV response uses UTF-8 with a BOM, an attachment filename and private/no-store headers. Cells beginning with a spreadsheet formula marker after leading whitespace, or starting with tab/newline/carriage return, are prefixed with an apostrophe. This helps prevent ordinary exported cell values from being interpreted as formulas.

Participation uses submissions divided by the current active student count, not class enrollment at the time of publication. Dashboard percentages are rounded to whole numbers and capped at 100; CSV participation uses two decimal places and is not capped. Feedback report averages use two decimal places; the displayed feedback overview uses one. An empty denominator gives zero participation.

8. Database and migration design

The complete data dictionary lists all 24 project models, all declared fields, lengths, defaults, blank/null behavior, relations, deletion rules, choices, constraints and explicit indexes.

Core relationships

User has one Profile and can connect to a Student, Faculty or Director record. Faculty are referenced by doubts, ratings and historical quizzes. Quiz contains Questions; each Question contains Options. A Submission references its Quiz and verified student User, and owns the original answer/detail rows with their selected Options.

Complaints retain the original AuthenticatedStudent and Teacher relationships. ComplaintEvent provides the conversation timeline. Resource references an uploader; Download references its resource and downloading user. Notification belongs to one recipient. Announcement has an author and a role audience.

Constraints and indexes

Rule	Purpose
Unique quiz code	Prevents two saved quizzes using the same generated lookup code
one_authenticated_quiz_attempt	At most one linked Submission for a quiz/student pair
one_authenticated_faculty_rating	At most one linked Rating for a student/faculty pair
rating_between_1_and_5	Database range check for faculty rating
Profile.role index	Supports role filters and recipient selection
Resource subject/category indexes	Supports library filtering
Complaint.status index	Supports status filtering
Notification recipient/read_at index	Supports unread count and recipient inbox queries

Null links retained for historical compatibility may allow multiple unlinked legacy records. They should not be read as verified authenticated student attempts or feedback ownership.

Migration history

Migrations 0001–0006 represent the original model history. Migration 0007 adds the secure academic fields and new modules. Migration 0008 converts accounts, hashes old passwords, assigns roles, links unambiguous records, fills quiz metadata/reviews and clears old sessions/OTPs. Migration 0009 removes plaintext password columns and adds uniqueness/rating constraints. Migration 0010 adds Student.batch and Submission.academic_details without guessing historical snapshots.

The conversion is intentionally irreversible. A rollback to an older schema requires restoring the matching backed-up database and application code, rather than unapplying the password-removal migration. Existing media paths and record IDs are retained by the intended upgrade path.

Historical ambiguity

Username conflicts do not silently take over an existing Django account. The migration can assign a unique conflict-safe username. The mapping command reports the resulting login names without passwords. Ambiguous email/enrollment matches remain unlinked instead of being assigned to an arbitrary student.

When repeated historical attempts/ratings map to the same authenticated pair, only the newest receives that link; older rows remain as historical data. This is why historical totals may differ from a clean new database. See UPGRADE.md for the detailed rehearsal and conflict-handling procedure.

9. Forms and validation

The forms reference lists every active form field, required flag, widget, choice and directly declared bound. AccountForm is listed separately for all three roles because the field sets differ.

The reference also includes the installed Django AuthenticationForm and PasswordChangeForm, plus three unsaved ProfileForm variants used by the current account/password workflows. Those forms are inspected with an unsaved dummy User, without loading any real account.

Browser attributes improve entry but do not replace server checks. The role decorator runs independently of whether a button is visible. ModelChoice faculty fields constrain the submitted target to active faculty users. The quiz-attempt view checks option ownership rather than trusting the radio/checkbox values emitted earlier.

The generic form template renders Django field errors and help text. For forms that accept files, POST must use multipart/form-data. The quiz builder also submits the management form for the questions formset; adding DOM fields without updating TOTAL_FORMS would not create a valid extra question.

Phone fields are strings with length bounds. Year and semester are descriptive text, not a validated academic calendar or cohort assignment. Email-uniqueness checks in forms should not be confused with a global database uniqueness constraint on Django User.email. Administrative/import flows have their own behavior and need review when extending account logic.

10. URLs and request contracts

The route reference lists every current project URL and original compatibility alias, the URL name, view, interface verbs, role/object access and fixed role arguments. It also covers Django account/password routes, query parameters and duplicate aliases.

Use reverse or Django’s url template tag for navigation. Avoid hardcoded endpoint strings in new templates when a named route exists. The project deliberately preserves case-sensitive historical paths such as /Director_dashboard/.

Response	Typical meaning in TeachTrack
200	Rendered page, attachment or health response
302	Login requirement, successful form redirect or navigation after a state change
400	Invalid Content-Length header handled by custom middleware

Response	Typical meaning in TeachTrack
403	Incorrect role, CSRF failure or explicit object permission denial
404	Missing/private object, unsupported kind or invalid selected account role
405	Wrong HTTP method on an explicitly restricted action
413	Declared body above the middleware's 12 MiB limit
429	Current username/IP throttle window is blocked

The application returns HTML forms and redirects. There is no documented bearer-token API, OAuth flow, CORS contract or JSON CRUD interface. POST actions use Django session cookies and CSRF protection.

11. Design, colours and front-end behavior

The design-system guide explains theme tokens, RGB values, semantic/status colours, typography, layout, spacing, borders, shadows, reusable components, breakpoints, images, icons and interaction states. The complete colour index includes every explicit hex/named colour from the active stylesheet with its selectors and properties. The offline HTML handbook adds searchable visual swatches.

Source of visual truth

ClassApp/static/ClassApp/app.css and theme.css are the active stylesheets. app.js handles application interactions; theme.js runs early to set the preferred theme before rendering. Legacy css/theme.css, jQuery and retained inline templates do not define current layouts.

The bottom-right sun/moon button switches every shared public/login/workspace layout. The preference is stored under teachtrack-theme, synchronized across tabs, and initially follows the device preference. Storage restrictions do not break the page. Theme tokens and hardcoded overrides are inventoried in COLOUR_INDEX.md, including scoped light/dark definitions.

Current JavaScript features

Interaction	Implementation
Sidebar location	Compares each sidebar link pathname with the current pathname; sets active class and aria-current
Mobile menu	Toggles the sidebar open class and the menu's aria-expanded value
Theme switch	Persists light/dark preference, follows initial system preference, and syncs tabs
Form submission	Shows progress, prevents duplicates and restores state on back navigation
Upload availability	Explains unavailable file uploads before submitting production forms
Escape key	Closes an open sidebar and resets aria-expanded
Add quiz question	Copies the empty-question template, replaces __prefix__, increments TOTAL_FORMS and stops at 50
Carousel arrows	Shows previous/next slide, wrapping between the last and first
Carousel dots	Selects a zero-based data-slide-to index and updates aria-current
Carousel keyboard	Left/Right arrows, Home and End while focus is within the carousel
Carousel touch	Horizontal touch movement over 50 pixels and larger than its vertical movement changes the slide

The carousel is manually controlled; it does not auto-advance. Exactly one slide is shown through the hidden attribute. The status text updates as Photo N of 7, and is announced through an aria-live region. New slides require a matching figure, dot, accessible count and initial counter text; the JavaScript's runtime total comes from the actual slides array.

Accessibility scope

The interface includes semantic navigation, labels, visible keyboard focus, alternative image text, status labels, form errors and keyboard-operable carousel buttons. All shared pages provide a keyboard skip link; reduced-motion preferences are respected. The mobile menu exposes expanded state. These are implementation features, not a completed independent accessibility certification. Colour combinations, small captions, screen-reader behavior and keyboard flow should be evaluated when changing the design.

12. Authentication, security and private files

Authentication and sessions

Django stores password hashes and authenticates active users through its normal backend. Password validators reject passwords under 10 characters, overly similar values, common passwords and numeric-only values. Password change requires the current password; public forgotten-password recovery and outbound email have been removed.

Session cookies are HTTP-only and SameSite=Lax. The configured age is 3,600 seconds and browser-close expiration is enabled. Secure session/CSRF cookies, HTTPS redirect and HSTS become enabled with DEBUG=False. Password changes use Django's session/auth-hash machinery; stored old plaintext values are not part of the current login flow.

Login throttling

Each POST hashes an ip:<peer-address> key and a user:<casefolded-username> key with SHA-256. Both counters are read with row locks in a transaction. Ten invalid attempts in either current window block authentication with HTTP 429 until the stored window is older than 15 minutes. The window is reset on a subsequent request after that threshold; it is not a sliding per-attempt timeout.

A successful sign-in removes the username counter. It does not reset accumulated IP failures, so several users sharing an IP can affect one another. The implementation reads REMOTE_ADDR and does not trust a client-supplied forwarded address. The real reverse-proxy behavior must be understood before relying on IP limits in a campus deployment.

Authorization and output handling

Role checks guard staff actions. Filtered querysets protect complaint/doubt/notification ownership. Quiz views check creator/attempt ownership and answer-release timing. Template autoescaping handles ordinary user-entered text; linebreaksbr is used to format some displayed content. Export cells receive the spreadsheet-formula prefix handling described earlier.

Authenticated responses receive private/no-store headers from the custom middleware. Download responses also set attachment delivery and private caching. Notification redirect targets must begin with one slash, not two, and contain no backslash; otherwise the endpoint returns to the inbox.

Upload formats and size policy

Kind	Validation performed
PDF	Extension .pdf and leading %PDF- bytes
TXT	UTF-8 decoding and absence of NUL characters
DOCX	ZIP contains word/document.xml and [Content_Types].xml

Kind	Validation performed
PPTX	ZIP contains ppt/presentation.xml and [Content_Types].xml
Office archive size	Sum of uncompressed entries must be below 50 MiB
JPG/JPEG/PNG where images are enabled	Pillow opens/verifies the image and permits JPEG/PNG formats; decompression-bomb warnings are rejected
File size	At most 10 × 1,024 × 1,024 bytes per validated upload
Declared request body	Custom middleware rejects more than 12 × 1,024 × 1,024 bytes

The resources form permits the document types above. Doubt attachments and announcement documents additionally permit validated JPEG/PNG images. QuestionEntryForm uses a Django ImageField plus the image-enabled validator. Existing image delivery recognizes selected stored image MIME types, but new uploads remain subject to the current form restrictions.

These checks validate recognizable formats and size; they do not perform antivirus scanning, strip active document content or prove a file harmless. Declared Content-Length checks do not independently enforce a streaming ingress limit when a client omits that header. Production ingress limits and any malware-scanning process are separate operations.

Private-media delivery

There is no public /media/ URL pattern. Resources use an authenticated POST download. Original doubt and announcement attachments use the attachments route with object/audience checks. Question images use a protected GET endpoint that checks quiz access. Production storage uses a private GCS backend and is read by the application before an authenticated FileResponse is returned.

Credential and data handling

Do not commit the private .env, .dev-secret, database dump, service-account file or other private credentials. The documentation uses placeholders. Backups of the original pre-upgrade database can contain plaintext credentials even though the current schema does not. User email, enrollment, feedback identity and complaint content remain personal application data in storage and exports; define appropriate campus access/retention policies before wider use.

13. Configuration and dependencies

The configuration reference documents every environment variable read by the project, its default and role, plus important non-environment Django settings and startup conditions.

Dependency responsibilities

Package declaration	Role in this project
Django==5.2.18	ORM, routing, templates, forms, sessions, authentication and admin
python-dotenv==1.2.4	Loads .env; runserver explicitly overrides inherited process values, other entry points do not
dj-database-url==3.1.2	Converts DATABASE_URL into Django database configuration
whitenoise==6.12.0	Serves compressed/hashed static files in the application process
psycopg[binary]==3.3.6	PostgreSQL database driver
gunicorn==26.2.0	Production WSGI process in the Linux container
django-storages[google]==1.14.6	Google Cloud Storage integration

Package declaration	Role in this project
google-cloud-storage==3.17.1	GCS client dependency
Pillow>=11.3,<13	Image parsing/validation; range declaration rather than an exact lock
mysql-connector-python	MySQL Connector driver, engine mysql.connector.django, installed by the main requirements

Requirements describe the intended install set. A virtual environment can differ if changed manually; inspect it with that environment's pip. Gunicorn is used by the Linux container; local Windows development uses Django runserver.

Database, storage and email selection

DATABASE_URL selects SQLite locally or PostgreSQL/MySQL; see CONFIGURATION_REFERENCE.md and VERCEL_DEPLOYMENT.md. Aiven uses MySQL Connector with certificate and hostname verification.

With DEBUG=True, blank GS_BUCKET_NAME uses local media storage. With DEBUG=False and no bucket, uploads are disabled at the form, middleware and storage layers. A private GCS bucket plus runtime credentials enables persistent uploads. A bucket is optional for production startup; text workflows continue without it. Supabase is not integrated.

Outbound email uses the dummy backend. SMTP variables, Forgot password, public reset links and email/phone OTP flows are removed. Signed-in users can change their password with the current password; trusted administrators can use Django changepassword from a private terminal.

14. Management commands

Command	Purpose	Data behavior
create_director	First/extra trusted director account	Prompts for a password, validates it, creates User/Profile and legacy director; refuses an existing username
seed_demo	Local sample workspace	Requires DEBUG=True and a strong DEMO_PASSWORD; refuses when any expected demo username already exists
account_migration_report	Legacy-to-current account mapping	Reports role record IDs/current usernames and unlinked historical counts; does not print passwords
import_legacy_students	Supported legacy SQLite student-table import	Read-only source; dry run by default; --apply creates eligible student accounts in target
changepassword	Django password reset from a trusted terminal	Prompts for a new password for an existing username
createsuperuser	Separate trusted Django administration	Creates a Django superuser; signal assigns director application role
migrate / showmigrations	Schema application/status	Changes or inspects the configured database; first review the upgrade path for populated data
collectstatic	Build current static manifest/output	Writes generated staticfiles/ content

Set an existing director password

```
$env:DEBUG = 'True'
```

```
.\.venv\Scripts\python.exe manage.py changepassword director
```

The password is entered in the terminal, not placed on the command line or in this documentation. Replace director with the actual username when different. A password change does not automatically clear the custom login-throttle window; a still-blocked login may need the normal cooldown.

Optional demo

```
$env:DEBUG = 'True'
$env:DEMO_PASSWORD = Read-Host "Choose a strong local demo password"
.\.venv\Scripts\python.exe manage.py seed_demo
Remove-Item Env:DEMO_PASSWORD
```

The resulting usernames are demo_director, demo_faculty, demo_student and demo_student2. They use the chosen password; no sample password is bundled. The command also creates resources, announcements, a quiz and answer records, ratings, a complaint timeline, a doubt and notifications. The built-in DEBUG guard prevents this command in the configured production mode.

Supported legacy import

```
.\.venv\Scripts\python.exe manage.py import_legacy_students "C:\legacy-copy\db.sqlite3"
```

The importer reads the specific ClassApp_student columns id, sname, sen, semail, unm and pw. It is not a complete faculty/quiz/complaint importer. It rejects source=target, username conflicts and invalid user data. Reviewed --apply creates current users with hashed passwords, retaining recognized Django hashes and setting unusable passwords when no old value exists. The source connection is explicitly closed for Windows file compatibility.

15. Testing and acceptance

Automated suite

There are 53 current test methods across WorkspaceTests and LegacyMigrationTests. The test catalogue lists each method and scenario. Tests cover role authorization, password/session behavior, forms, private uploads/downloads, quiz grading/availability/duplicate protection, feedback constraints, complaint/doubt access, account management, export handling and legacy data migration.

Isolated Windows test run

Use process-level SQLite configuration in a temporary shell scope so the verification is isolated from an existing MySQL workspace:

```
$previousDatabaseUrl = $env:DATABASE_URL
$previousDebug = $env:DEBUG
try {
    $env:DEBUG = 'True'
    $env:DATABASE_URL = 'sqlite:///:memory:'
    .\.venv\Scripts\python.exe manage.py test ClassApp --noinput
} finally {
    if ($null -eq $previousDatabaseUrl) { Remove-Item Env:DATABASE_URL -ErrorAction SilentlyContinue } else { $env:DATABASE_URL =
$previousDatabaseUrl }
```

```
    if ($null -eq $previousDebug) { Remove-Item Env:DEBUG -ErrorAction SilentlyContinue } else { $env:DEBUG = $previousDebug }
}
```

The migration test separately creates a temporary database matching the old schema, populates historical records and applies the upgrade. It launches the same interpreter as the test process and checks preserved records, password conversion, relationships, duplicates and constraints. The normal TestCase suite uses separate test storage and in-memory media configuration.

Other checks

```
$env:DEBUG = 'True'
.\.venv\Scripts\python.exe manage.py check
.\.venv\Scripts\python.exe manage.py makemigrations --check --dry-run
.\.venv\Scripts\python.exe manage.py collectstatic --noinput
node --check ClassApp/static/ClassApp/app.js
.\.venv\Scripts\python.exe tools/build_documentation.py
```

Node is only needed for the optional JavaScript syntax check or browser-verification tooling; it is not required to run this Django project. Documentation generation needs the installed Python/Django environment but no additional Markdown/PDF Python library.

Browser acceptance

Local Microsoft Edge browser checks covered public pages at 390/768/1440 pixels, role login/navigation, director faculty creation, student quiz autofill/submission/result grid, keyboard skip navigation and the visible persistent light/dark toggle. These observations do not certify a protected Vercel deployment, Safari/Firefox, real mobile hardware or production latency. See WEBSITE_REVIEW.md for verified checks and limits.

16. Deployment, maintenance and recovery

Container behavior

The Dockerfile uses python:3.12-slim, installs the main requirements, collects static files with a build-only development setting, creates a non-root app user and starts Gunicorn. The default bind is 0.0.0.0 on PORT or 8080, with 2 workers, 4 threads and a 120-second timeout. Access/error logs go to standard output/error.

Migrations are separate from web startup. This avoids having several web instances race to migrate at the same time. The supplied requirements install PostgreSQL and MySQL Connector support; runtime connection settings and the matching certificate are still required.

Production startup requirements

With DEBUG=False, settings require SECRET_KEY of at least 50 characters and a PostgreSQL or MySQL configuration. GS_BUCKET_NAME is optional; omitting it disables uploads.

On Vercel, DEBUG defaults false and supplied VERCEL_URL, VERCEL_PROJECT_PRODUCTION_URL and VERCEL_BRANCH_URL hostnames are added to ALLOWED_HOSTS and HTTPS CSRF origins. Custom domains still belong in explicit configuration. Process environment wins over .env except local manage.py runserver, which deliberately loads .env with override=True. See VERCEL_DEPLOYMENT.md for deploy and verification steps.

Release process

1. Review source and migration changes; back up the existing database and uploaded files.

2. Run checks and the isolated test suite against the intended release source.
3. Build the container/static output and apply necessary migrations through the controlled migration job.
4. Deploy the matching application image with the intended runtime secrets/configuration.
5. Verify login, roles, SQL connectivity, theme assets, own profile, enabled private object download, quiz submission/result layout, complaint updates and reports.
6. Record the release version, configuration changes, migration state and verification results.

Health and logging

/health/ returns plain-text ok. It is a process-level health endpoint; it does not query the database or prove database or cloud-bucket readiness. Django logging uses console output at INFO. Local server output can be redirected to the runserver log files, but those files are not a formal audit trail or a full monitoring system.

Backups and recovery

Database backups and media backups form a pair. A database row referencing a file is insufficient if the file was not backed up. For local SQLite, use a consistent backup procedure when writes are stopped or a supported SQLite backup mechanism; for MySQL/PostgreSQL use the engine's proper dump/snapshot tooling. Restore into a separate test target first and validate accounts, old media, submissions and complaint timelines.

An ordinary code-only rollback and a migration rollback are different operations. Do not unapply the irreversible account/password conversion as a recovery shortcut. For that upgrade family, restore the corresponding pre-upgrade database and old code as a matched set.

17. Troubleshooting

Symptom	Likely cause	What to do
Login says credentials are incorrect	Email entered instead of username, wrong password, absent account or inactive user	Use the actual Django username; check account status; use signed-in password change or a trusted terminal administrator
Faculty login offers no public staff creation	Faculty accounts are director-managed	Sign in as director and use Manage accounts → Faculty
Faculty form shows year/semester	Old page/cached source or student role selected	Open current account form, choose Faculty and refresh; the active staff form removes these fields
Correct login is still blocked	IP/username failure window reached 10 attempts	Observe the normal 15-minute window; password change alone does not clear counters
SECRET_KEY required in production during local run	Inherited DEBUG is not one of the accepted true values	Set $env:DEBUG = 'True' in the local shell; inspect process configuration without printing secrets
Django cannot be imported	Wrong interpreter or missing installed requirements	Run pip and manage.py with the same .venv interpreter
py -3.12 cannot find Python	That exact interpreter is not installed	Use an installed supported project interpreter; do not replace a working environment just to activate it
MySQL driver/server error	MySQL Connector absent, server/certificate unavailable or DATABASE_URL incorrect	Install requirements.txt and check the private URL and matching public CA
No such table / inconsistent history	Migrations absent, wrong database or incomplete historical SQLite file	Inspect migration status and use the appropriate fresh/upgrade procedure; do not fake a mismatched schema
DisallowedHost	Requested hostname not in ALLOWED_HOSTS	Configure that hostname without a URL scheme

Symptom	Likely cause	What to do
CSRF failure	Missing token, changed origin, secure-cookie or proxy mismatch	Verify form token, exact HTTPS trusted origin and trusted-proxy setup
HTTP 403/404 on a private object	Role/ownership checks or missing file	Use an authorized account and confirm the object/file exists
HTTP 405 downloading a file	Download endpoints require POST	Use the supplied Download form rather than browsing the URL directly
Image displays alternative text	Missing asset, wrong static path, failed request or stale output	Inspect the image response; edit source assets and regenerate collectstatic output
Homepage stays unchanged	Cached page/assets or an older server process	Refresh with Ctrl+F5 and make sure the intended server is serving the port
Carousel arrows/dots do not move	app.js failed to load or invalid slide markup	Check script response/console and data-carousel/data-slide/dot structure
Forgot password is absent	Public password reset and SMTP were removed	Signed-in users use Password; trusted administrators can run changepassword from a private terminal
Upload rejected	Unsupported format, invalid content, image/archive limit or size	Use a permitted validated file within the documented bounds
Quiz not open	Unpublished, upcoming, closed or invalid question structure	Check publication, IST dates and correct-option definitions
Another quiz attempt is refused	An authenticated attempt already exists	Open that saved result; there is no reset/re-attempt UI
No faculty in a selection	No active faculty-role account exists	Create/activate faculty through the director’s account-management page

18. Scope, limitations and extension guide

Current limits

This version has one campus workspace, without tenant isolation, class/cohort enrollment, attendance, fees, timetable or a full gradebook. Quizzes have one authenticated attempt, exact-set auto-grading, no short-answer evaluator, no partial marks, no per-user countdown and no question-weight configuration. Resource assignment files are learning uploads; the app does not provide a separate assignment hand-in/marking workflow.

An own-account profile editor is available; there are no user-facing role-change, resource-edit/delete, quiz-edit/delete or notification-delete workflows in the current routes. Contact messages are stored without an automatic email workflow. Announcements, notifications and feedback are ordinary server-rendered data, not live push streams. API/mobile clients, multi-factor authentication and institutional SSO are not implemented.

Legacy historical data can affect totals. Feedback is identity-hidden in its template rather than anonymized in storage. A full malware-scanning pipeline, streaming ingress policy, abuse controls for the public contact form, load testing and independent accessibility/security reviews remain separate development/operational work.

Extending the application consistently

Change	Files and checks to consider
New model field	models.py, a new migration, form validation, views, templates, historical defaults and generated database reference

Change	Files and checks to consider
New protected action	Named route, role decorator, filtered object queryset, CSRF/method enforcement, UI affordance and permission tests
New account field	Student/staff field differences, Profile/legacy persistence, bootstrap/import handling and validation
New resource category	Resource.CATEGORIES, migration if applicable, form/filter labels, UI and exports
New quiz question behavior	Form structure, valid_quiz, attempt validation/scoring, review snapshot, answer tables and grading tests
New announcement audience	Choices, permitted actor roles, recipient selection, listing/download access and notification fan-out
New colour/theme	Tokens, literal usages, semantic/status meaning, focus/contrast checks and rebuilt colour catalogue
New carousel photo	Source image, figure/alt text/count, dot, initial count and mobile-width checks
New campus fact/image	Verify the official source, update CAMPUS_CONTENT.md and regenerate static output

Keep the original migration files unchanged. Add new migrations after the existing chain. Regenerate references with tools/build_documentation.py after a schema/form/route/colour change. Update authored explanations when behavior changes; generation does not infer the product meaning of new logic.

Documentation maintenance

Edit the authored handbook, design/configuration guides, RECENT_CHANGES.md and deployment/testing notes when behavior changes. Then run tools/build_documentation.py, tools/export_documentation_pdf.py and tools/check_documentation.py from the project root.

The builder isolates Django from production with in-memory SQLite and a build-only secret. It reads metadata and unsaved form variants, not account rows. Generated outputs include model/form/route/test/colour references, JSON tokens, diagrams and the combined offline handbook. The exporter refreshes the PDF using an optional local browser with external HTTP blocked. The checker validates maintained local links and current feature references. See tools/README.md for usage.

19. Glossary

Term	Meaning in this project
Faculty	Teaching/staff application role; also the original Faculty contact model
Director	Campus administration role in Profile, separate from Django superuser status
PYQ	Previous-year question paper resource category
Doubt	Student question assigned to a faculty account
Profile	Per-user role/department/enrollment metadata
Legacy record	Original academic/contact record retained through the upgrade
Wrapper	AuthenticatedStudent or Teacher record used by original complaint relationships
ORM	Django’s model/query interface to the configured relational database
Migration	Versioned schema/data change applied through Django management commands
CSRF	Form-request protection tied to the expected browser/site context

Term	Meaning in this project
WSGI / ASGI	Python web-application entry-point interfaces; current container uses WSGI
Static asset	Version-controlled CSS, JavaScript, fonts or presentation images
Media	User-uploaded private file stored locally or in the configured bucket
Manifest	Generated mapping from source static names to hash-named collected output
M2M	Many-to-many relationship, implemented through a join table
Snapshot	Saved text/identity/answer values at a point in time, such as quiz review JSON
IST	India Standard Time; project timezone Asia/Kolkata
MiB	1,024 × 1,024 bytes; the code’s file/body limits use this binary multiplier

TeachTrack design system

Current shared UI updates

The active stylesheets are app.css and theme.css; the colour inventory now includes both, with selector scope retained for light/dark tokens. theme.js initializes the saved browser preference before rendering; app.js handles menus, forms and quizzes. The sun/moon button is fixed at bottom-right across public, login, dashboard and error templates. It has an accessible label, pressed state and keyboard focus.

Result academic details use an aligned grid: five columns on large screens, three at 900px and two at 480px. Labels sit above values with no browser default dd indent. Question numbers are inside cards above question text; the attempt form is at most 800px wide and long text/options wrap. Launch/deadline labels explicitly show IST.

The public Contact page retains the homepage header/footer. All shared pages have a keyboard skip link; reduced-motion preferences are respected. Mobile controls, form actions and toolbar wrapping are adjusted at small widths. Form navigation shows progress and prevents duplicate submission; back navigation restores it.

Light muted text uses #596b75. Dark mode sets the page background to #10191f, surfaces to #1a2831, primary text to #e6edf1 and muted text to #a5b5be. The generated colour index is authoritative for exact selector/property declarations.

Design reference for the source reviewed on 21 October 2026. The active visual implementation is ClassApp/static/ClassApp/app.css plus theme.css, loaded by base.html. This guide explains the design decisions expressed by that stylesheet; it does not introduce an additional theme framework.

1. Visual direction and source files

TeachTrack uses a restrained teal-and-green palette, light surfaces, dark text, rounded cards and compact academic controls. The public pages have more open space and large introductory typography. Authenticated screens prioritize navigation, filtering, forms, tables and readable status feedback.

Source	Responsibility
ClassApp/static/ClassApp/app.css	Current tokens, layouts, components, states and breakpoints
ClassApp/static/ClassApp/app.js	Sidebar, mobile navigation, quiz formset and carousel behavior
Templates/base.html	Common HTML head, application shell and authentication shell
Templates/public_base.html	Public website header, navigation, main area and footer
Templates/index.html	Homepage introduction, seven-photo carousel and feature cards
Templates/campus_life.html	Campus identity, cover photograph, facilities, gallery and location
Templates/registration/login.html	Current role-aware sign-in screens
Templates/account_form.html	Director-managed account role links and role-specific fields
ClassApp/static/img/	Source photography and image assets

The retained original theme and legacy template inline styles are not the active design reference. The generated staticfiles/ directory is build output. Make a design change in source and regenerate collected assets rather than editing a hash-named output file.

2. Colour index and theme variables

The active stylesheet contains 111 distinct explicit hex literals, plus named white and transparent values. COLOUR_INDEX.md lists every value, RGB triplet, alpha and direct selector/property usage. colour-index.json provides the same data for tools. The offline HTML handbook includes visual swatches and a filter for colour values or selectors.

Base tokens

Token	Value	RGB	Main purpose
--bg	#f5f7fa	245, 247, 250	Default application body background
--ink	#172d35	23, 45, 53	Main headings and body text
--muted	#596b75	89, 107, 117	Secondary descriptions, help text and small metadata
--line	#e5ebee	229, 235, 238	General borders and dividers
--teal	#126c64	18, 108, 100	Primary buttons, links, accent text and selected states
--teal-dark	#0e514b	14, 81, 75	Hover/darker primary interaction colour
--pale	#e8f4ef	232, 244, 239	Selected navigation and pale accent surfaces
--white	#fff	255, 255, 255	Standard panels, input and secondary-button surfaces
--shadow	0 3px 12px #19353c04	25, 53, 60 at 1.57% alpha	Very subtle application-card shadow

These are CSS custom properties defined in :root. They are inherited globally. Most functional components use them, but public/campus components also contain explicit green/grey values. A token change does not automatically recolour those literal values.

Main public/campus palette

Value	Role	Example selectors
#f7f9f7	Public page canvas	.public-site
#143c35	Deep green public footer	.site-footer
#d3e0d9	General footer foreground	.site-footer
#c4d8cb	Footer navigation links	.site-footer nav a
#a7c1b0	Footer bottom metadata	.footer-bottom
#d7e7dc	Footer brand-icon surface	.site-footer .brand-icon
#204c3e	Footer brand-icon letter	.site-footer .brand-icon
#dfe8e2	Public dividers	.home-features; .campus-spaces; .campus-location-tag
#edf3ee	Campus facility-card surface	.campus-facilities article
#e6f0e8	Campus location panel surface	.campus-visit
#193e35	Carousel base and portrait-image mat	.carousel-stage; .campus-slide-portrait img

Value	Role	Example selectors
#c6d4ca	Inactive carousel dot	.carousel-dots button::after
#76a88b	Small campus-marker dot	.campus-marker
#e7af49	Keyboard-focus outline	:focus-visible selectors

The detailed source index is authoritative for all usages. For example, #dfe8e2 remains on several current separators even though the homepage's former home-community element and its CSS were removed.

State and meaning colours

State	Background	Foreground	Selector / use
Pending complaint / selected incorrect review	#fff3dc	#9a6b17	.pending
In-progress complaint	#eaf0ff	#5474ae	.in_progress
Resolved complaint / correct review	#e7f4ec	#3c8261	.resolved
General success/info alert	#e8f4eb	#36744f	.alert; border #cbe5d2
Error alert	#fff1ed	#a7523e	.alert.error; border #efd5ce
Field error list	#fff0ed	#ac4239	.errorlist
Unread notification	#f0f8f3	Normal content colours	.notification.unread; left border #65a98d
Default resource/file icon	#eef5f1	#3c8d70	.file-icon
Every-third list-row icon	#fff5e6	#bd8c3a	.list-row:nth-child(3n) .file-icon
Participation progress	#edf3ef track	#539b84 fill	progress and browser-specific progress selectors
Readable answer block	#f0f7f3	Inherited ink	.answer-box

The resource icon variation is positional CSS, not a fixed Notes/PYQ/Assignment colour mapping. Status meaning is expressed in text as well as colour. The same pending/resolved styling is reused in quiz review, so colour interpretation depends on the displayed label.

Alpha colours and overlays

Literal	RGB	Alpha	Purpose
#fffffff5	255, 255, 255	96.08%	Public sticky header surface
#ffffffc9	255, 255, 255	78.82%	Authenticated topbar surface
#fffffff2	255, 255, 255	94.90%	Carousel arrow surface
#ffffff80	255, 255, 255	50.20%	Carousel arrow border
#ffffff20	255, 255, 255	12.55%	Public footer separator
#102b24e8	16, 43, 36	90.98%	Dark end of photo-caption gradient
#122d2526	18, 45, 37	14.90%	Carousel arrow shadow
#17303322	23, 48, 51	13.33%	Mobile sidebar shadow

Literal	RGB	Alpha	Purpose
#173e3512	23, 62, 53	7.06%	Carousel shadow
#1c493214	28, 73, 50	7.84%	Campus cover shadow

Eight-digit values use RRGGBBAA. Alpha is the final byte divided by 255. The visual result is a composite with whatever is underneath; the RGB value alone is not the final on-screen colour. The swatch catalogue shows transparency against a checkerboard to make this distinction visible.

Contrast measurements

The values below were computed from the sRGB relative-luminance formula for the listed opaque pairs. They describe these pairs only; they are not an audit of every rendered element, transparency composite or text size.

Foreground / background	Ratio
#172d35 on #ffffff	14.35 : 1
#6e7e86 on #ffffff	4.21 : 1
#ffffff on #126c64	6.26 : 1
#ffffff on #0e514b	9.14 : 1
#d3e0d9 on #143c35	8.93 : 1
#9a6b17 on #fff3dc	4.26 : 1
#5474ae on #eaf0ff	4.10 : 1
#3c8261 on #e7f4ec	4.07 : 1
#a7523e on #fff1ed	4.86 : 1

Some muted/status combinations have substantially less contrast than the main text and buttons. Evaluate their actual font size, weight and surrounding surface when setting an accessibility target. The project does not claim that all small text or colour combinations have independently passed such a review.

3. Typography

The active stylesheet uses a system font stack: system-ui, -apple-system, BlinkMacSystemFont, Segoe UI, sans-serif. The current interface does not request a hosted web font. Retained original font assets do not change that stack unless a template explicitly loads the older theme.

Element	Declared size / style
Body	14px, line-height 1.65
Generic h1	32px, weight 700, line-height 1.25, letter-spacing -1px
Generic h2	18px, line-height 1.4, letter-spacing -0.3px
Generic h3	15px
Small text	12px
Eyebrow	10px, weight 750, 1.9px letter-spacing
Standard form label	12px, weight 600

Element	Declared size / style
Help text	11px, muted, block layout
Error list	12px, coloured surface and rounded edges
Default table body	12px
Default table headings	10px, uppercase, 0.6px letter-spacing
Navigation	Sidebar 12px; public header links 12px
Main number in stat card	33px with a compact line-height
Homepage hero	clamp(44px, 4.5vw, 62px), line-height 1.1
Campus hero	clamp(42px, 4.5vw, 62px), line-height 1.08
Authentication hero	49px before responsive overrides
Sign-in card heading	24px before responsive overrides
Carousel caption heading	25px; 21px in the smallest breakpoint
Campus space-card heading	22px; 21px in the smallest breakpoint

Responsive rules deliberately replace some clamp values with fixed sizes after the layout changes. Homepage hero text becomes 48px below 1050px, 55px in the stacked layout below 800px and 47px below 480px. Campus hero text becomes 51px below 950px, 54px below 700px and 46px below 480px. Do not infer final rendered size from the first rule alone.

4. Layout and spacing

Public layout

The public content container has a maximum width of 1180px. Its declared width is viewport width minus 80px, changing to minus 48px at 1050px and minus 36px at 480px. This corresponds to 40px, 24px and 18px side margins when the max-width does not otherwise constrain it.

The desktop public header has an 88px minimum height, a white translucent surface and a bottom border. It is sticky at the viewport top with z-index 30. Below 800px, brand/account actions stay on the upper row while navigation moves to its own full-width row. Below 480px, the separate Get started header button is hidden; student registration remains available elsewhere.

Homepage hero uses a 1fr/1.12fr grid with a 56px gap. Below 800px it stacks. The feature grid has three columns on desktop and one in the compact public layout. The public footer uses a 1.6fr/1fr/1fr grid, then two columns at small phone widths with the introduction spanning both columns.

Authenticated layout

The sidebar is fixed, full-height and initially 242px wide. Main content uses a matching left margin and calculated width. At 1180px the sidebar becomes 215px; at 900px it becomes 200px. At 680px it is hidden until opened by the menu button, and the content fills the viewport. The open mobile sidebar is 242px wide with a shadow.

The application topbar is 77px high, reducing to 66px on phones. Main padding starts at 35px 36px 20px and becomes smaller at tablet/phone widths. Stat cards use four columns, then two. General two-column panels stack at 900px. The resource grid uses three, two or one column according to the general app breakpoints.

Campus detail layout

The campus hero pairs flexible text with a maximum 399px cover column. At 950px the cover column becomes 330px; below 700px it stacks and centers a cover up to 370px. The cover image’s natural portrait ratio is preserved so its Campus title remains visible.

Library/café cards use two columns, stacking below 700px. Facilities use four columns, two below 950px and one below 480px. The photo gallery uses three columns, two below 700px and one below 480px. The location panel pairs text with a 372px image column, reducing to 300px below 950px and stacking below 700px.

Spacing conventions

The project uses explicit pixel spacing, not a comprehensive named spacing-token scale. Common values include 8–12px for label/icon gaps, 14–20px for control/list spacing, 22–28px for card content, 35–40px for larger sections and 55–70px for public-page separation. Preserve the surrounding component’s rhythm when adding a new element instead of assuming every gap must be a multiple of one global unit.

5. Borders, radii and shadows

Element	Treatment
Standard button	1px teal border; 8px radius
Form input/select/textarea	1px #d6e1e3 border; 7px radius; 11px 12px padding
General panel/stat card	1px light border; 11px radius; subtle --shadow
Public feature card	12px radius and #e0e8e2 border
Carousel	18px radius, hidden overflow and deeper soft shadow; 12px radius on phones
Carousel arrows	Circular, 42px diameter; 36px at the smallest breakpoint
Carousel dot buttons	28px wide and 32px high; visible mark is 7px, selected mark 23px
Campus cover	8px white frame, 16px outer radius and 9px image radius
Campus gallery tile	11px radius and light border
Campus visit panel	14px radius and pale green surface
Status badge	5px radius with 4px 9px padding
Answer block	7px radius and 19px padding
Keyboard focus	3px amber outline with 3px offset on standard interactive elements

Use component classes for meaning rather than duplicating inline styles. The CSS contains border/shadow literals where a component needs a distinct treatment; the colour index records those values and exact selectors.

6. Component reference

Component	Main selectors	Behavior / purpose
Brand	.brand, .brand-icon, .brand-dot	Text wordmark and code-rendered T tile; footer uses a lighter tile
Primary action	button, .button	Filled teal with white text; darker hover
Secondary action	.secondary	White surface, neutral border; pale teal hover

Component	Main selectors	Behavior / purpose
Small/full action	.small, .full	Compact padding/text or full-width button
Sidebar	.sidebar, .workspace, .nav-badge	Fixed module navigation, group label and unread count
Account identity	.sidebar-bottom, .avatar	Name/role summary and first-letter avatar
Topbar	.topbar, .top-actions	Notifications, password change and POST sign-out
Page heading	.page-head, .head-actions	Page title/description with contextual actions
Summary numbers	.stats, .stat-card	Role-dependent dashboard values
Welcome panel	.welcome-banner, .banner-art	Decorative dashboard introduction and quiz link
General panel	.panel, .section-head	Content card with optional heading/action
Resource rows/cards	.list-row, .resource-card, .resource-bottom	Resource metadata, file symbol and download action
Filtering	.filter-bar	Search/category/subject/sort controls with responsive wrapping
Form	.form-panel, .form-actions, .helptext, .errorlist	Labelled fields, validation and save/navigation actions
Role links	.role-switch	GET navigation between sign-in or account-creation roles; current item is teal
Question builder	.question-form, .question-image	Formset question groups and bounded images
Quiz choices	.option, .option:has(input:checked)	Clickable labels; selected option uses a pale surface and teal border
Status	.status, .pending, .in_progress, .resolved	Textual semantic state with colour
Alerts	.alert, .alert.error	Messages and errors announced through template status/alert roles
Table	.table-wrap, table, th, td	Compact tabular information; horizontal wrapper when needed
Inbox	.notification, .notification.unread	Read/unread styling and recipient actions
Complaint timeline	.timeline	Vertical line, event markers and chronological conversation
Answer/result	.answer-box, .answer-row, .score	Faculty answers, option review and score display
Charts	.chart-row, progress, .rating-chart, meter	Native HTML progress/meter visualization, not an external chart package
Pagination	.pagination	Previous/next page and page context
Authentication	.auth-main, .auth-layout, .auth-intro, .auth-card	Anonymous brand, intro panel and sign-in form
Public header/footer	.site-header, .site-nav, .site-footer	Shared public navigation and campus/account links
Homepage carousel	.campus-carousel, .carousel-stage, .campus-slide	User-controlled photos, captions and controls
Campus identity	.campus-identity, .campus-cover	Institute logo and full portrait main image
Campus details	.campus-spaces-grid, .campus-facilities, .campus-photo-grid, .campus-visit	Facilities, student-life gallery and contact/location

Button and control states

Primary hover changes to --teal-dark and stays white. Secondary hover uses --pale with teal text/border. Links normally underline on hover, while button-like links suppress that underline. Standard links, buttons, inputs, textareas and selects receive a focus-visible outline. Checkbox/radio accents use --teal. Selected role links and carousel dots use aria-current-driven selectors.

Hidden carousel figures use the native hidden attribute; there is no fade/slide animation or autoplay timer. The question builder disables Add question at 50 forms. A disabled add control is not a replacement for the formset’s server-side maximum.

7. Complete responsive breakpoint index

Query	Main effects
min-width:1500px	Larger authenticated main padding and stat gaps
max-width:1180px	Smaller sidebar, general app padding and two-column resource grid
max-width:1050px	Public container margins/header gaps and homepage hero adjustments
max-width:950px	Campus hero/visit image columns shrink; facilities become two columns
max-width:900px	Smaller sidebar, two-column stats, stacked general two-column panels and stacked authentication layout
max-width:800px	Public header navigation wraps; homepage stacks; public features become one column
max-width:700px	Campus hero, space cards and visit panel stack; gallery becomes two columns
max-width:680px	Authenticated mobile sidebar/menu, full-width main, one-column resource grid and compact app controls
max-width:480px	Public phone layout, one-column campus facilities/gallery, smaller headings/arrows and two-column footer

These breakpoints apply to different component families. Do not consolidate them without testing the affected screens. Public pages and application screens do not share every layout threshold.

8. Photography and asset index

Homepage carousel

Photo	Source file	Content	Natural size
1	Slider-img4.jpg	Students studying on campus lawn	1600 × 1066
2	Slider-img1.jpg	Students and guitar in common area	1600 × 1066
3	Slider-img2.jpg	Student panel discussion; portrait mat	864 × 1152
4	Slider-img3.jpg	Student/faculty group	1600 × 1200
5	Slider-img5.jpg	Auditorium gathering	1080 × 890
6	sait-campus-extra-1.jpg	Institute building and lawn	559 × 357
7	sait-campus-extra-2.jpg	Computer lab	447 × 447

Carousel images normally use object-fit:cover and a 4:3 frame. The portrait slide uses contain to preserve the image. The stacked tablet layout uses a 3:2 frame before returning to 4:3 on the smallest phones. The bottom caption uses a transparent-to-dark gradient. White circular arrows remain visible over the photo.

Campus page images

File	Purpose	Natural size
sait-logo.jpg	Institute identity	447 × 447
sait-campus-main.jpg	Selected main cover; full image without cropping	399 × 501
about-img2.jpg	Library/Knowledge Resource Center card	1080 × 654
about-img1.jpg	Common Ground card	1080 × 693
sait-industry-visit.jpg	Student-life gallery	678 × 452
sait-campus-building.jpg	Location panel	372 × 299
Seven carousel files above	Student-life gallery; combined with industry-visit image gives eight photos	See above

The supplied thumbnails have limited native resolution. They are stored unchanged; the app does not claim to reconstruct high-resolution originals. Keep the selected main image at an appropriate size and preserve its visible title. Source URLs are in CAMPUS_CONTENT.md. The older Banner-bg.jpg remains an available original asset but is not the selected current cover.

Icons

The current shell uses Unicode symbols for several module icons and small inline SVG paths for carousel arrows. SVG arrows inherit currentColor. Retained FontAwesome assets exist, but the current base template does not load that old icon stylesheet. Photographic logos are images rather than generated typography.

9. Accessibility and content conventions

Use real buttons for actions and links for navigation. Keep visible text alongside status colours. Provide useful alt text that describes the photograph rather than its filename. Use a native hidden state for inactive carousel slides so hidden content does not remain an ordinary visible page region.

Public pages include a skip link. The sidebar menu exposes aria-controls and aria-expanded. The carousel exposes labels, selected-dot state, a polite live status and keyboard controls. Headings, form labels, field errors and table headings provide structure. Keep arrow and dot hit areas usable when adding photos; the current seven-dot layout was checked at 320px and 390px widths.

Student year/semester fields belong to student forms, not staff forms. Faculty sign-in should explain director-managed account creation rather than direct a new staff user to public student registration. Campus descriptions should use verified official information with source links. Keep internal implementation details out of ordinary product prompts unless they help the person complete the task.

10. Safe design-change workflow

1. Identify the active template and class rather than editing a retained legacy file with a similar name.
2. Locate the token or literal in app.css; use the complete colour index to identify related usages.
3. Implement the component change in source, including hover/focus/selected/error states where relevant.
4. Verify the surrounding public/app layout at affected breakpoints, including narrow phones.

5. Verify real image responses and the browser console for interactive changes.
6. Run collectstatic for current generated output and rebuild documentation when values/components change.

No blanket colour replacement should be treated as a finished redesign. Check status meaning, text contrast, SVG inheritance, alpha overlays and screenshots after changing the palette.

Database reference

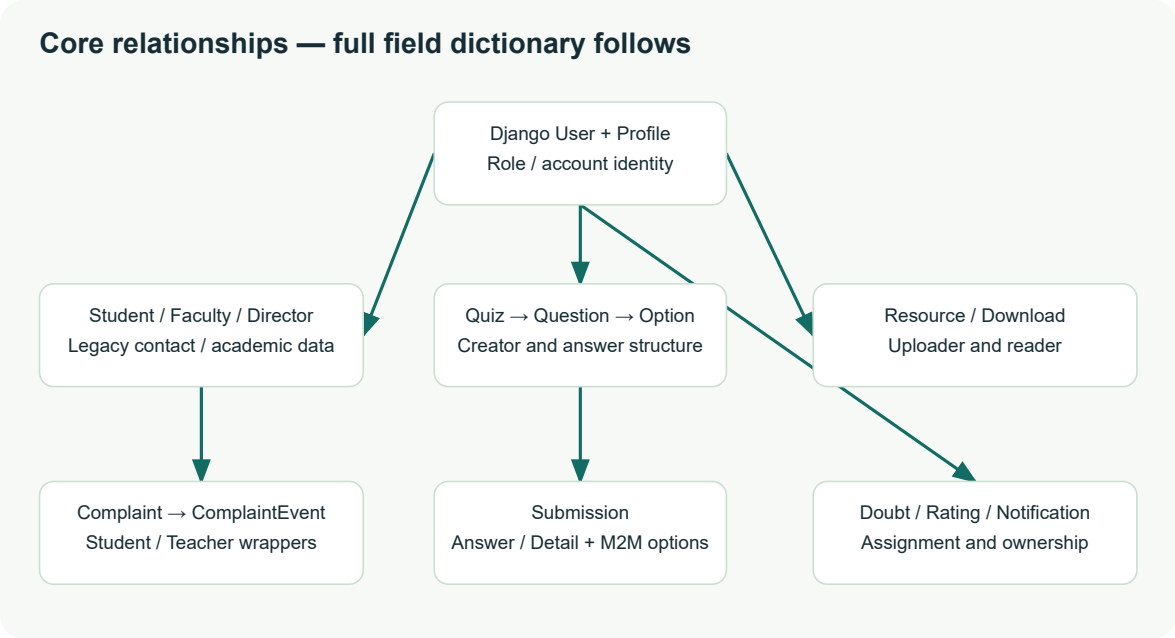
Extracted from Django model metadata on 21 October 2026.

This is the current declared schema. The installed database must also have migrations 0001–0009 applied. Nullable legacy relationships are intentional and do not authorize an arbitrary user to view a historical record.

Reading the dictionary

FK means foreign key; O2O means one-to-one; M2M means many-to-many. `blank` is form/model-validation behavior, `null` permits SQL NULL, and `PROTECT` prevents deletion while referenced. `CASCADE` removes dependent rows; `SET_NULL` retains the row and clears the link. M2M fields use separate join tables.

Django authentication, groups, permissions, sessions, content types and admin-log tables are provided by installed Django applications. This reference lists every project model and its declared fields, including automatically generated IDs.



Student

Original student academic record. Its Django user link connects legacy details to current authentication.

Table: `ClassApp_student`. Default ordering: `database default`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.
`sname`	CharField(100)	—	null=False; blank=False	—	Student full name.
`sen`	CharField(50)	—	null=False; blank=False	—	Student enrollment number.
`sphone`	CharField(10)	—	null=False; blank=False	—	Student phone text; maximum 10 characters.
`semail`	CharField(254)	—	null=False; blank=False	—	Student email address.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`unm`	CharField(50)	—	null=False; blank=False	—	Retained legacy username value; current login uses Django User.username.
`course`	CharField(50)	''	null=False; blank=False	—	Student course/department text.
`year`	CharField(20)	''	null=False; blank=False	—	Student academic year text.
`sem`	CharField(20)	''	null=False; blank=False	—	Legacy student semester value.
`section`	CharField(5)	''	null=False; blank=False	—	Student section text.
`batch`	CharField(30)	''	null=False; blank=True	—	Student batch/cohort text, editable at registration and in My profile.
`user`	OneToOneField	—	unique; indexed; null=True; blank=True	auth.User; PROTECT	Link to the related record.

Faculty

Original faculty contact record; also the assignment target for doubts and feedback.

Table: `ClassApp_faculty`. Default ordering: `database default`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.
`fname`	CharField(100)	—	null=False; blank=False	—	Faculty full name.
`fphone`	CharField(15)	—	null=False; blank=False	—	Faculty phone text; maximum 15 characters.
`femail`	CharField(254)	—	unique; null=True; blank=True	—	Faculty email; nullable for retained legacy records.
`unm`	CharField(50)	—	unique; null=False; blank=False	—	Retained legacy username value; current login uses Django User.username.
`user`	OneToOneField	—	unique; indexed; null=True; blank=True	auth.User; PROTECT	Link to the related record.

Director

Original director contact record. Application authorization is determined by Profile.role.

Table: `ClassApp_director`. Default ordering: `database default`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.
`dname`	CharField(100)	—	null=False; blank=False	—	Director full name.
`dphone`	CharField(15)	—	null=False; blank=False	—	Director phone text; maximum 15 characters.
`demail`	CharField(254)	—	unique; null=False; blank=False	—	Director email address.
`unm`	CharField(50)	—	unique; null=False; blank=False	—	Retained legacy username value; current login uses Django User.username.
`user`	OneToOneField	—	unique; indexed; null=True; blank=True	auth.User; PROTECT	Link to the related record.

Profile

One profile for every Django user; stores the application role, department and enrollment.

Table: `ClassApp_profile`. Default ordering: `database default`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.
`user`	OneToOneField	—	unique; indexed; null=False; blank=False	auth.User; CASCADE	Link to the related record.
`role`	CharField(10)	Profile.Role.STUDENT	indexed; null=False; blank=False; choices: student, faculty, director	—	student, faculty or director; authorization reads this value.
`enrollment`	CharField(90)	—	null=False; blank=True	—	Enrollment metadata; required by student registration forms.
`department`	CharField(100)	—	null=False; blank=True	—	Course/department metadata.

Contact

Messages submitted through the public TeachTrack contact form. Saving a message does not send an email.

Table: `ClassApp_contact`. Default ordering: `database default`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.
`name`	CharField(100)	—	null=False; blank=False	—	Name.
`email`	CharField(254)	—	null=False; blank=False	—	Email.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`title`	CharField(200)	—	null=False; blank=False	—	Title.
`message`	TextField	—	null=False; blank=False	—	Message.
`created_at`	DateTimeField	Automatic	null=False; blank=True	—	Creation timestamp, set automatically.

ImageUploader

Retained historical image model. No current public upload workflow routes to it.

Table: `ClassApp_imageuploader`. Default ordering: `database default`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.
`photo`	FileField(100)	—	null=False; blank=False	—	Photo. Upload path: ALLImages.
`date`	DateTimeField	now()	null=False; blank=False	—	Date.

PasswordResetOTP

Retained historical OTP table. Public password reset and outbound email are disabled.

Table: `ClassApp_passwordresetotp`. Default ordering: `database default`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.
`email`	CharField(254)	—	null=False; blank=False	—	Email.
`otp`	CharField(6)	—	null=False; blank=False	—	Historical reset OTP text; no current public OTP/reset workflow.
`created_at`	DateTimeField	Automatic	null=False; blank=True	—	Creation timestamp, set automatically.

Quiz

Quiz metadata, creator, availability, publication flag and student-detail requirements.

Table: `ClassApp_quiz`. Default ordering: `['-created_at', '-id']`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`title`	CharField(255)	—	null=False; blank=False	—	Title.
`created_at`	DateTimeField	Automatic	null=False; blank=True	—	Creation timestamp, set automatically.
`expiry_time`	DateTimeField	—	null=False; blank=False	—	Quiz closing timestamp; attempts require current time strictly before this value.
`is_active`	BooleanField	True	null=False; blank=False	—	Quiz publication/visibility flag, separate from its time window.
`require_name`	BooleanField	False	null=False; blank=False	—	Whether the quiz-attempt name field is mandatory.
`require_roll`	BooleanField	False	null=False; blank=False	—	Whether the quiz-attempt roll/enrollment field is mandatory.
`require_email`	BooleanField	False	null=False; blank=False	—	Whether the quiz-attempt email field is mandatory.
`require_class`	BooleanField	False	null=False; blank=False	—	Whether the quiz-attempt course/class field is mandatory. New quiz forms enable this automatically.
`faculty`	ForeignKey	—	indexed; null=True; blank=True	ClassApp.Faculty; CASCADE	Link to the related record.
`code`	CharField(6)	—	unique; null=True; blank=True	—	Six uppercase hexadecimal characters, generated when a quiz is saved without a code.
`subject`	CharField(100)	—	null=False; blank=True	—	Subject.
`starts_at`	DateTimeField	now()	null=False; blank=False	—	Quiz opening timestamp; the opening instant itself is eligible.
`creator`	ForeignKey	—	indexed; null=True; blank=True	auth.User; PROTECT	Link to the related record.

Question

A quiz question, optional image and single-selection/multiple-selection setting.

Table: `ClassApp_question`. Default ordering: `['id']`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.
`quiz`	ForeignKey	—	indexed; null=False; blank=False	ClassApp.Quiz; CASCADE	Link to the related record.
`text`	TextField	—	null=False; blank=False	—	Text.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`image`	FileField(100)	—	null=True; blank=True	—	Image. Upload path: quiz_images/.
`is_multiple`	BooleanField	False	null=False; blank=False	—	True enables selection of multiple correct options.

Option

An answer option and its correctness flag, attached to one question.

Table: `ClassApp_option`. Default ordering: `['id']`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.
`question`	ForeignKey	—	indexed; null=False; blank=False	ClassApp.Question; CASCADE	Link to the related record.
`text`	CharField(255)	—	null=False; blank=False	—	Text.
`is_correct`	BooleanField	False	null=False; blank=False	—	Correct-answer flag; students do not receive it before the review is released.

Submission

A recorded quiz attempt with identity and academic snapshots, aggregate score and a saved grading review.

Table: `ClassApp_submission`. Default ordering: `['-score', 'submitted_at', 'id']`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.
`quiz`	ForeignKey	—	indexed; null=False; blank=False	ClassApp.Quiz; CASCADE	Link to the related record.
`submitted_at`	DateTimeField	Automatic	null=False; blank=True	—	Submission timestamp, set automatically.
`student_name`	CharField(255)	—	null=True; blank=True	—	Student display-name snapshot, including historical records.
`roll_no`	CharField(50)	—	null=True; blank=True	—	Roll/enrollment value copied into an attempt.
`email`	CharField(254)	—	null=True; blank=True	—	Email.
`class_div`	CharField(50)	—	null=True; blank=True	—	Legacy combined academic summary, limited to 50 characters; full separate values are in academic_details.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`score`	IntegerField	0	null=False; blank=False	—	Correct-answer count; on SubmissionDetail this is 0 or 1 for one question.
`student`	ForeignKey	—	indexed; null=True; blank=True	auth.User; PROTECT	Link to the related record.
`total`	PositiveIntegerField	0	null=False; blank=False	—	Number of questions in the saved attempt.
`review`	JSONField	list()	null=False; blank=False	—	Saved question/option text and selected/correct answer IDs used by the results view.
`academic_details`	JSONField	dict()	null=False; blank=True	—	Submission JSON snapshot containing course, year, semester, section and batch separately.

Declared constraints:

- `one_authenticated_quiz_attempt`: UniqueConstraint — quiz, student.

SubmissionAnswer

Original per-question answer relationship, including the selected Option many-to-many relation.

Table: `ClassApp_submissionanswer`. Default ordering: `database default`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.
`submission`	ForeignKey	—	indexed; null=False; blank=False	ClassApp.Submission; CASCADE	Link to the related record.
`question`	ForeignKey	—	indexed; null=False; blank=False	ClassApp.Question; CASCADE	Link to the related record.
`selected_options`	ManyToManyField	—	null=False; blank=False	ClassApp.Option; M2M via ClassApp_submissionanswer_selected_options	Selected answers, stored through an automatically created join table.

SubmissionDetail

Original detailed answer record, retained alongside SubmissionAnswer for historical compatibility.

Table: `ClassApp_submissiondetail`. Default ordering: `database default`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`quiz`	ForeignKey	—	indexed; null=False; blank=False	ClassApp.Quiz; CASCADE	Link to the related record.
`submission`	ForeignKey	—	indexed; null=False; blank=False	ClassApp.Submission; CASCADE	Link to the related record.
`question`	ForeignKey	—	indexed; null=False; blank=False	ClassApp.Question; CASCADE	Link to the related record.
`student_name`	CharField(255)	—	null=True; blank=True	—	Student display-name snapshot, including historical records.
`roll_no`	CharField(50)	—	null=True; blank=True	—	Roll/enrollment value copied into an attempt.
`email`	CharField(254)	—	null=True; blank=True	—	Email.
`class_div`	CharField(50)	—	null=True; blank=True	—	Legacy combined academic summary, limited to 50 characters; full separate values are in academic_details.
`score`	IntegerField	0	null=False; blank=False	—	Correct-answer count; on SubmissionDetail this is 0 or 1 for one question.
`selected_options`	ManyToManyField	—	null=False; blank=False	ClassApp.Option; M2M via ClassApp_submissiondetail_selected_options	Selected answers, stored through an automatically created join table.

Doubt

A student question assigned to faculty, with an optional attachment and an editable faculty answer.

Table: `ClassApp_doubt`. Default ordering: `['-submitted_at', '-id']`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.
`student_name`	CharField(100)	—	null=False; blank=False	—	Student display-name snapshot, including historical records.
`student_email`	CharField(254)	—	null=False; blank=False	—	Student email snapshot, including historical records.
`doubt`	TextField	—	null=False; blank=False	—	Doubt.
`attachment`	FileField(100)	—	null=True; blank=True	—	Attachment. Upload path: attachments/.
`faculty`	ForeignKey	—	indexed; null=False; blank=False	ClassApp.Faculty; CASCADE	Link to the related record.
`submitted_at`	DateTimeField	Automatic	null=False; blank=True	—	Submission timestamp, set automatically.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`student`	ForeignKey	—	indexed; null=True; blank=True	auth.User; PROTECT	Link to the related record.
`subject`	CharField(100)	—	null=False; blank=True	—	Subject.
`answer`	TextField	—	null=False; blank=True	—	Latest faculty answer text; the form allows an existing answer to be updated.
`answered_at`	DateTimeField	—	null=True; blank=True	—	Time of the most recent saved faculty answer.

Announcement

A staff-authored update for a selected role or everyone, optionally with a document.

Table: `ClassApp_announcement`. Default ordering: `['-created_at', '-id']`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.
`title`	CharField(200)	—	null=False; blank=False	—	Title.
`content`	TextField	—	null=False; blank=False	—	Content.
`posted_by`	ForeignKey	—	indexed; null=True; blank=True	ClassApp.Faculty; SET_NULL	Link to the related record.
`document`	FileField(100)	—	null=True; blank=True	—	Document. Upload path: announcements/.
`created_at`	DateTimeField	Automatic	null=False; blank=True	—	Creation timestamp, set automatically.
`author`	ForeignKey	—	indexed; null=True; blank=True	auth.User; PROTECT	Link to the related record.
`audience`	CharField(10)	'all'	null=False; blank=False; choices: all, student, faculty, director	—	all, student, faculty or director; faculty forms restrict selection to student.

Rating

Student feedback about faculty. A student can update their existing rating and comment.

Table: `ClassApp_rating`. Default ordering: `['-updated_at', '-id']`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.
`student_name`	CharField(100)	—	null=False; blank=False	—	Student display-name snapshot, including historical records.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`student_email`	CharField(254)	—	null=False; blank=False	—	Student email snapshot, including historical records.
`rating`	PositiveSmallIntegerField	—	null=False; blank=False	—	Integer rating from 1 to 5, with form and database constraints.
`faculty`	ForeignKey	—	indexed; null=False; blank=False	ClassApp.Faculty; CASCADE	Link to the related record.
`submitted_at`	DateTimeField	Automatic	null=False; blank=True	—	Submission timestamp, set automatically.
`student`	ForeignKey	—	indexed; null=True; blank=True	auth.User; CASCADE	Link to the related record.
`comment`	TextField	—	null=False; blank=True	—	Comment.
`updated_at`	DateTimeField	now()	null=False; blank=False	—	Timestamp updated by the relevant application workflow; it is not auto_now.

Declared constraints:

- `one_authenticated_faculty_rating` : UniqueConstraint — student, faculty.
- `rating_between_1_and_5` : CheckConstraint — (AND: ('rating__gte', 1), ('rating__lte', 5)).

AuthenticatedStudent

Original complaint-system student wrapper, linked to a Django user.

Table: `ClassApp_authenticatedstudent`. Default ordering: `database default`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.
`user`	OneToOneField	—	unique; indexed; null=False; blank=False	auth.User; CASCADE	Link to the related record.
`name`	CharField(100)	—	null=False; blank=False	—	Name.

Teacher

Original complaint-system faculty wrapper, linked to a Django user.

Table: `ClassApp_teacher`. Default ordering: `database default`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`user`	OneToOneField	—	unique; indexed; null=False; blank=False	auth.User; CASCADE	Link to the related record.
`name`	CharField(100)	—	null=False; blank=False	—	Name.

Complaint

A student concern assigned to a teacher, with a current status and latest faculty reply.

Table: `ClassApp_complaint`. Default ordering: `['-updated_at', '-id']`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.
`student`	ForeignKey	—	indexed; null=False; blank=False	ClassApp.AuthenticatedStudent; CASCADE	Link to the related record.
`teacher`	ForeignKey	—	indexed; null=False; blank=False	ClassApp.Teacher; CASCADE	Link to the related record.
`text`	TextField	—	null=False; blank=False	—	Text.
`teacher_reply`	TextField	—	null=True; blank=True	—	Latest faculty reply; full conversation is stored in ComplaintEvent.
`status`	CharField(20)	'pending'	indexed; null=False; blank=False; choices: pending, in_progress, resolved	—	Complaint state: pending, in_progress or resolved.
`created_at`	DateTimeField	Automatic	null=False; blank=True	—	Creation timestamp, set automatically.
`reply_time`	DateTimeField	—	null=True; blank=True	—	Time of the latest faculty reply.
`title`	CharField(180)	'Student concern'	null=False; blank=False	—	Title.
`updated_at`	DateTimeField	now()	null=False; blank=False	—	Timestamp updated by the relevant application workflow; it is not auto_now.

ComplaintEvent

Chronological complaint messages and the status recorded for each event.

Table: `ClassApp_complaintevent`. Default ordering: `['created_at', 'id']`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.
`complaint`	ForeignKey	—	indexed; null=False; blank=False	ClassApp.Complaint; CASCADE	Link to the related record.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`author`	ForeignKey	—	indexed; null=False; blank=False	auth.User; PROTECT	Link to the related record.
`message`	TextField	—	null=False; blank=False	—	Message.
`status`	CharField(20)	—	null=False; blank=False; choices: pending, in_progress, resolved	—	Complaint state: pending, in_progress or resolved.
`created_at`	DateTimeField	Automatic	null=False; blank=True	—	Creation timestamp, set automatically.

Resource

A private notes, PYQ or assignment upload with descriptive metadata and download count.

Table: `ClassApp_resource`. Default ordering: `['-created_at', '-id']`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.
`title`	CharField(180)	—	null=False; blank=False	—	Title.
`description`	TextField	—	null=False; blank=True	—	Description.
`subject`	CharField(100)	—	indexed; null=False; blank=False	—	Subject.
`category`	CharField(12)	—	indexed; null=False; blank=False; choices: notes, pyq, assignment	—	Resource category: notes, pyq or assignment.
`file`	FileField(255)	—	null=False; blank=False	—	File. Upload path: resource_path().
`original_name`	CharField(255)	—	null=False; blank=False	—	Uploaded filename after removing directory components; used for attachment downloads.
`uploaded_by`	ForeignKey	—	indexed; null=False; blank=False	auth.User; PROTECT	Link to the related record.
`download_count`	PositiveIntegerField	0	null=False; blank=False	—	Download-response initiations, including repeats; not proof of completed transfers.
`created_at`	DateTimeField	Automatic	null=False; blank=True	—	Creation timestamp, set automatically.

Download

An authenticated resource-download initiation. Repeat downloads create additional rows.

Table: `ClassApp_download`. Default ordering: `database default`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.
`resource`	ForeignKey	—	indexed; null=False; blank=False	ClassApp.Resource; CASCADE	Link to the related record.
`user`	ForeignKey	—	indexed; null=False; blank=False	auth.User; CASCADE	Link to the related record.
`created_at`	DateTimeField	Automatic	null=False; blank=True	—	Creation timestamp, set automatically.

Notification

An individual recipient inbox item. Broadcasts create a separate row for each recipient.

Table: `ClassApp_notification`. Default ordering: `['-created_at', '-id']`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.
`recipient`	ForeignKey	—	indexed; null=False; blank=False	auth.User; CASCADE	Link to the related record.
`title`	CharField(180)	—	null=False; blank=False	—	Title.
`message`	TextField	—	null=False; blank=True	—	Message.
`link`	CharField(255)	—	null=False; blank=True	—	Optional notification navigation target; the read endpoint only redirects to safe local paths.
`read_at`	DateTimeField	—	null=True; blank=True	—	Null until the recipient marks this notification as read.
`created_at`	DateTimeField	Automatic	null=False; blank=True	—	Creation timestamp, set automatically.

Explicit indexes: `ClassApp_no_recipie_b833fa_idx` on `['recipient', 'read_at']`.

LoginThrottle

Hashed username/IP failure counters for the 15-minute login-attempt window.

Table: `ClassApp_loginthrottle`. Default ordering: `database default`.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`id`	BigAutoField	—	primary key; null=False; blank=True	—	Automatically generated primary key.

Field	Type / length	Default	Rules	Relation / deletion	Meaning
`key`	CharField(64)	—	unique; null=False; blank=False	—	SHA-256 digest of an ip: or user: throttle key; not a plaintext password.
`failures`	PositiveIntegerField	0	null=False; blank=False	—	Invalid authentication attempts counted in the current window.
`window_start`	DateTimeField	—	null=False; blank=False	—	Beginning of the stored throttle window.

Forms and validation reference

Extracted from active form classes. Required marks below apply to an unbound form; quiz-attempt identity fields become required according to that quiz's four requirement flags.

Public student registration ignores injected role or superuser fields. Director-only account routes determine the selected role on the server and remove student-only academic fields for staff. Phone inputs limit character length; they do not implement a numeric-only or international telephone-number validator.

StudentRegistrationForm

Field	Label	Field type	Required	Widget	Validation / choices
`username`	Username	UsernameField	Yes	TextInput	max_length=150
`first_name`	Full name	CharField	Yes	TextInput	max_length=100
`email`	Email	EmailField	Yes	EmailInput	max_length=320
`enrollment`	Enrollment	CharField	Yes	TextInput	max_length=50
`department`	Course / department	CharField	Yes	TextInput	max_length=50
`phone`	Phone	CharField	No	TextInput	max_length=10
`year`	Year	CharField	No	TextInput	max_length=20
`semester`	Semester	CharField	No	TextInput	max_length=20
`section`	Section	CharField	No	TextInput	max_length=5
`batch`	Batch	CharField	No	TextInput	max_length=30
`password1`	Password	CharField	Yes	PasswordInput	Default Django field validation
`password2`	Password confirmation	CharField	Yes	PasswordInput	Default Django field validation

AccountForm — student

Field	Label	Field type	Required	Widget	Validation / choices
`username`	Username	UsernameField	Yes	TextInput	max_length=150
`first_name`	Full name	CharField	Yes	TextInput	max_length=100
`email`	Email	EmailField	Yes	EmailInput	max_length=320
`enrollment`	Enrollment	CharField	Yes	TextInput	max_length=50
`department`	Course / department	CharField	Yes	TextInput	max_length=50
`phone`	Phone number	CharField	No	TextInput	max_length=10
`year`	Year	CharField	No	TextInput	max_length=20
`semester`	Semester	CharField	No	TextInput	max_length=20
`section`	Section	CharField	No	TextInput	max_length=5

Field	Label	Field type	Required	Widget	Validation / choices
`batch`	Batch	CharField	No	TextInput	max_length=30
`role`	Role	ChoiceField	Yes	HiddenInput	choices: student, faculty, director; hidden input
`password1`	Password	CharField	Yes	PasswordInput	Default Django field validation
`password2`	Password confirmation	CharField	Yes	PasswordInput	Default Django field validation

AccountForm — faculty

Field	Label	Field type	Required	Widget	Validation / choices
`username`	Username	UsernameField	Yes	TextInput	max_length=150
`first_name`	Full name	CharField	Yes	TextInput	max_length=100
`email`	Email	EmailField	Yes	EmailInput	max_length=320
`department`	Department	CharField	Yes	TextInput	max_length=50
`phone`	Phone number	CharField	No	TextInput	max_length=15
`role`	Role	ChoiceField	Yes	HiddenInput	choices: student, faculty, director; hidden input
`password1`	Password	CharField	Yes	PasswordInput	Default Django field validation
`password2`	Password confirmation	CharField	Yes	PasswordInput	Default Django field validation

AccountForm — director

Field	Label	Field type	Required	Widget	Validation / choices
`username`	Username	UsernameField	Yes	TextInput	max_length=150
`first_name`	Full name	CharField	Yes	TextInput	max_length=100
`email`	Email	EmailField	Yes	EmailInput	max_length=320
`department`	Department	CharField	Yes	TextInput	max_length=50
`phone`	Phone number	CharField	No	TextInput	max_length=15
`role`	Role	ChoiceField	Yes	HiddenInput	choices: student, faculty, director; hidden input
`password1`	Password	CharField	Yes	PasswordInput	Default Django field validation
`password2`	Password confirmation	CharField	Yes	PasswordInput	Default Django field validation

ResourceForm

Field	Label	Field type	Required	Widget	Validation / choices
`title`	Title	CharField	Yes	TextInput	max_length=180
`subject`	Subject	CharField	Yes	TextInput	max_length=100
`category`	Category	TypedChoiceField	Yes	Select	choices: , notes, pyq, assignment
`description`	Description	CharField	No	Textarea	Default Django field validation
`file`	File	FileField	Yes	ClearableFileInput	max_length=255

QuizForm

Field	Label	Field type	Required	Widget	Validation / choices
`title`	Title	CharField	Yes	TextInput	max_length=255
`subject`	Subject	CharField	No	TextInput	max_length=100
`starts_at`	Opens at (IST)	DateTimeField	Yes	DateTimeInput	Default Django field validation
`expiry_time`	Closes at (IST)	DateTimeField	Yes	DateTimeInput	Default Django field validation

QuestionEntryForm

Field	Label	Field type	Required	Widget	Validation / choices
`text`	Question	CharField	Yes	Textarea	max_length=5000
`image`	Question image (JPG/PNG, optional)	ImageField	No	ClearableFileInput	Default Django field validation
`is_multiple`	Allow multiple correct options	BooleanField	No	CheckboxInput	Default Django field validation
`option_a`	Option a	CharField	Yes	TextInput	max_length=255
`option_b`	Option b	CharField	Yes	TextInput	max_length=255
`option_c`	Option c	CharField	No	TextInput	max_length=255
`option_d`	Option d	CharField	No	TextInput	max_length=255
`correct`	Correct	MultipleChoiceField	Yes	CheckboxSelectMultiple	choices: A, B, C, D
`DELETE`	Remove this question	BooleanField	No	CheckboxInput	Default Django field validation

StudentInfoForm

Field	Label	Field type	Required	Widget	Validation / choices
`student_name`	Full name	CharField	No	TextInput	max_length=255
`roll_no`	Roll / enrollment number	CharField	No	TextInput	max_length=50
`email`	Email	EmailField	No	EmailInput	max_length=320
`course`	Course / class	CharField	No	TextInput	max_length=50
`year`	Year	CharField	No	TextInput	max_length=20
`semester`	Semester	CharField	No	TextInput	max_length=20
`section`	Section	CharField	No	TextInput	max_length=5
`batch`	Batch	CharField	No	TextInput	max_length=30

AnnouncementForm

Field	Label	Field type	Required	Widget	Validation / choices
`title`	Title	CharField	Yes	TextInput	max_length=200
`content`	Content	CharField	Yes	Textarea	Default Django field validation
`document`	Document	FileField	No	ClearableFileInput	max_length=100
`audience`	Audience	TypedChoiceField	Yes	Select	choices: all, student, faculty, director

ComplaintForm

Field	Label	Field type	Required	Widget	Validation / choices
`faculty`	Faculty	ModelChoiceField	Yes	Select	Authorized model choices; see workflow rules
`title`	Title	CharField	Yes	TextInput	max_length=180
`body`	Body	CharField	Yes	Textarea	max_length=10000

DoubtForm

Field	Label	Field type	Required	Widget	Validation / choices
`faculty`	Faculty	ModelChoiceField	Yes	Select	Authorized model choices; see workflow rules
`subject`	Subject	CharField	No	TextInput	max_length=100

Field	Label	Field type	Required	Widget	Validation / choices
`doubt`	Doubt	CharField	Yes	Textarea	Default Django field validation
`attachment`	Attachment	FileField	No	ClearableFileInput	max_length=100

FeedbackForm

Field	Label	Field type	Required	Widget	Validation / choices
`faculty`	Faculty	ModelChoiceField	Yes	Select	Authorized model choices; see workflow rules
`rating`	Rating	IntegerField	Yes	Select	min_value=0
`comment`	Comment	CharField	No	Textarea	Default Django field validation

ReplyForm

Field	Label	Field type	Required	Widget	Validation / choices
`message`	Message	CharField	Yes	Textarea	max_length=5000
`status`	Status	ChoiceField	No	Select	choices: pending, in_progress, resolved

AnswerForm

Field	Label	Field type	Required	Widget	Validation / choices
`answer`	Answer	CharField	Yes	Textarea	max_length=5000

NotificationForm

Field	Label	Field type	Required	Widget	Validation / choices
`title`	Title	CharField	Yes	TextInput	max_length=180
`message`	Message	CharField	Yes	Textarea	max_length=5000
`audience`	Audience	ChoiceField	Yes	Select	choices: all, student, faculty, director

ContactForm

Field	Label	Field type	Required	Widget	Validation / choices
`name`	Name	CharField	Yes	TextInput	max_length=100
`email`	Email	EmailField	Yes	EmailInput	max_length=254
`title`	Title	CharField	Yes	TextInput	max_length=200
`message`	Message	CharField	Yes	Textarea	Default Django field validation

ProfileForm — student

Field	Label	Field type	Required	Widget	Validation / choices
`username`	Username	CharField	Yes	TextInput	max_length=150
`first_name`	Full name	CharField	Yes	TextInput	max_length=100
`email`	Email	EmailField	Yes	EmailInput	max_length=320
`department`	Department	CharField	Yes	TextInput	max_length=50
`phone`	Phone number	CharField	No	TextInput	max_length=10
`enrollment`	Enrollment	CharField	No	TextInput	max_length=50
`year`	Year	CharField	No	TextInput	max_length=20
`semester`	Semester	CharField	No	TextInput	max_length=20
`section`	Section	CharField	No	TextInput	max_length=5
`batch`	Batch	CharField	No	TextInput	max_length=30

ProfileForm — faculty

Field	Label	Field type	Required	Widget	Validation / choices
`username`	Username	CharField	Yes	TextInput	max_length=150
`first_name`	Full name	CharField	Yes	TextInput	max_length=100
`email`	Email	EmailField	Yes	EmailInput	max_length=320
`department`	Department	CharField	Yes	TextInput	max_length=50
`phone`	Phone number	CharField	No	TextInput	max_length=15

ProfileForm — director

Field	Label	Field type	Required	Widget	Validation / choices
`username`	Username	CharField	Yes	TextInput	max_length=150

Field	Label	Field type	Required	Widget	Validation / choices
`first_name`	Full name	CharField	Yes	TextInput	max_length=100
`email`	Email	EmailField	Yes	EmailInput	max_length=320
`department`	Department	CharField	Yes	TextInput	max_length=50
`phone`	Phone number	CharField	No	TextInput	max_length=15

Django AuthenticationForm

Field	Label	Field type	Required	Widget	Validation / choices
`username`	Username	UsernameField	Yes	TextInput	max_length=150
`password`	Password	CharField	Yes	PasswordInput	Default Django field validation

Django PasswordChangeForm

Field	Label	Field type	Required	Widget	Validation / choices
`old_password`	Old password	CharField	Yes	PasswordInput	Default Django field validation
`new_password1`	New password	CharField	Yes	PasswordInput	Default Django field validation
`new_password2`	New password confirmation	CharField	Yes	PasswordInput	Default Django field validation

Additional server rules

- Email uniqueness is checked case-insensitively in the account form; the email is stored lowercase. Django User.email itself is not made globally unique by a database constraint.
- Passwords use Django’s configured similarity, minimum-length (10), common-password and numeric-only rejection validators.
- The quiz closing time must be later than the opening time.
- QuestionFormSet requires 1–50 question forms and uses the `questions` prefix. Options A and B are mandatory; C and D are optional.
- A single-selection question needs exactly one marked correct option. Every marked correct option needs text.
- Multiple-selection questions are also checked by valid_quiz before students can attempt them; a valid quiz needs at least one correct option per question.
- Complaint, doubt and feedback faculty choices include only active faculty users with the faculty role, ordered by name then ID.
- Faculty announcement and notification forms permit the Students audience only; directors may choose all configured audiences.
- A complaint reply requires a message up to 5,000 characters. Only directors receive the status input. A complaint body accepts up to 10,000 characters.
- Resource, announcement and doubt file validation and image safety checks are detailed in the handbook Security and uploads chapter.
- New quiz forms automatically publish and require name, enrollment, email and course; these flags are not displayed.
- StudentInfoForm autofills identity and separate course/year/semester/section/batch fields from the account.

- ProfileForm edits only the authenticated account and synchronizes linked role records; roles and permissions cannot be edited here.
- Login and signed-in password change use Django forms. Public password reset, phone OTP and SMTP are disabled.

Route reference

Paths are relative to the application host. Local example: `http://127.0.0.1:8000/`.

The Methods column describes verbs used by the interface. Explicit method decorators enforce POST for mutations such as downloads, mark-read, publication toggles and account deactivation; the quiz-attempt view explicitly permits GET/POST. Other read views do not all reject alternative verbs. This is a server-rendered HTML application; these URLs are not a JSON REST API.

`pk` and `submission_id` are integer identifiers. `kind` selects an allowed attachment or report type. An authenticated user with an incorrect role receives a permission error; missing/foreign private objects may return 404.

Current and compatibility routes

Path	URL name	View	Methods	Access	Fixed arguments
`/accounts/profile/`	`profile_edit`	profile_edit	GET, POST	Authenticated; own account only	—
`/dashboard/`	`dashboard`	dashboard	GET	Authenticated	—
`/`	`index`	index	GET	Public	—
`/index/`	`—`	index	GET	Public	—
`/contact/`	`contact`	contact	GET, POST	Public	—
`/health/`	`health`	health	GET	Public	—
`/campus-life/`	`campus_life`	campus_life	GET	Public	—
`/question-images/<int:pk>/`	`question_image`	question_image	GET	Authenticated; quiz access checks	—
`/attachments/<str:kind>/<int:pk>/`	`attachment_download`	attachment_download	POST	Authenticated; object/audience checks	—
`/submission-details/`	`submission_detail_list`	submission_detail_list	GET	Own student; creator faculty; director	—
`/resources/`	`resources`	resources	GET	Authenticated	—
`/resources/upload/`	`resource_create`	resource_create	GET, POST	Faculty or director	—
`/resources/<int:pk>/download/`	`resource_download`	resource_download	POST	Authenticated	—
`/notifications/`	`notifications`	notifications	GET	Own inbox	—
`/notifications/send/`	`notification_create`	notification_create	GET, POST	Faculty or director	—
`/notifications/read-all/`	`notifications_read_all`	notifications_read_all	POST	Own inbox	—
`/notifications/<int:pk>/read/`	`notification_read`	notification_read	POST	Own notification	—
`/announcements/`	`announcements`	announcements	GET	Authenticated; audience filtered	—
`/announcements/create/`	`announcement_create`	announcement_create	GET, POST	Faculty or director	—

Path	URL name	View	Methods	Access	Fixed arguments
`/quizzes/`	`quizzes`	quizzes	GET	Authenticated; role filtered	—
`/quizzes/create/`	`quiz_create`	quiz_create	GET, POST	Faculty or director	—
`/quizzes/<int:pk>/toggle/`	`quiz_toggle`	quiz_toggle	POST	Creator faculty or director	—
`/quizzes/<int:pk>/attempt/`	`quiz_attempt`	quiz_attempt	GET, POST	Student; open published valid quiz	—
`/quizzes/<int:pk>/result/`	`quiz_result`	quiz_result	GET	Own student attempt; creator faculty; director	—
`/quizzes/<int:pk>/leaderboard/`	`leaderboard`	leaderboard	GET	Authenticated; publication/ownership checks	—
`/complaints/`	`complaints`	complaints	GET	Own student; assigned faculty; director	—
`/complaints/new/`	`complaint_create`	complaint_create	GET, POST	Student	—
`/complaints/<int:pk>/`	`complaint_detail`	complaint_detail	GET, POST	Own student; assigned faculty; director	—
`/doubts/`	`doubts`	doubts	GET	Own student; assigned faculty; director	—
`/doubts/new/`	`doubt_create`	doubt_create	GET, POST	Student	—
`/doubts/<int:pk>/answer/`	`doubt_answer`	doubt_answer	GET, POST	Assigned faculty only	—
`/feedback/`	`feedback`	feedback	GET, POST	Authenticated; only students can POST	—
`/accounts/manage/`	`accounts`	accounts	GET	Director	—
`/accounts/manage/new/`	`account_create`	account_create	GET, POST	Director	—
`/accounts/manage/<int:pk>/toggle/`	`account_toggle`	account_toggle	POST	Director	—
`/reports/`	`reports`	reports	GET	Director	—
`/reports/<str:kind>/export/`	`report_export`	report_export	GET	Director	—
`/student-registration/`	`student_registration`	register	GET, POST	Public; creates students only	—
`/faculty-registration/`	`faculty_registration`	account_create	GET, POST	Director	role=faculty
`/director-registration/`	`director_registration`	account_create	GET, POST	Director	role=director
`/student-login/`	`student_login`	sign_in	GET, POST	Public; optional route-specific role	role=student
`/faculty-login/`	`faculty_login`	sign_in	GET, POST	Public; optional route-specific role	role=faculty
`/director-login/`	`director_login`	sign_in	GET, POST	Public; optional route-specific role	role=director

Path	URL name	View	Methods	Access	Fixed arguments
`/student_dashboard/`	`student_dashboard`	student_dashboard	GET	Student	—
`/faculty_dashboard/`	`faculty_dashboard`	faculty_dashboard	GET	Faculty	—
`/Director_dashboard/`	`Director_dashboard`	Director_dashboard	GET	Director	—
`/create-quiz/`	`create_quiz`	quiz_create	GET, POST	Faculty or director	—
`/teacher/`	`teacher_quiz_list`	quizzes	GET	Authenticated; role filtered	—
`/student/`	`student_quiz_list`	quizzes	GET	Authenticated; role filtered	—
`/attempt/<int:pk>/`	`attempt_quiz`	quiz_attempt	GET, POST	Student; open published valid quiz	—
`/submit/<int:pk>/`	`submit_quiz`	quiz_attempt	GET, POST	Student; open published valid quiz	—
`/submissions/<int:pk>/`	`quiz_submissions`	quiz_result	GET	Own student attempt; creator faculty; director	—
`/score/<int:submission_id>/`	`view_score`	legacy_score	GET	Own student; creator faculty; director	—
`/doubt/`	`doubt`	doubt_create	GET, POST	Student	—
`/doubt-viewer/`	`doubt_viewer`	doubts	GET	Own student; assigned faculty; director	—
`/rating/`	`rating`	feedback	GET, POST	Authenticated; only students can POST	—
`/announcements/add/`	`add_announcement`	announcement_create	GET, POST	Faculty or director	—
`/announcements/`	`announcement`	announcements	GET	Authenticated; audience filtered	—
`/complaints/add/`	`add_complaint`	complaint_create	GET, POST	Student	—
`/complaints/`	`complaints_list`	complaints	GET	Own student; assigned faculty; director	—
`/complaints/reply/<int:pk>/`	`teacher_reply`	complaint_detail	GET, POST	Own student; assigned faculty; director	—

Authentication and administration

Path	URL name	Methods	Access
`/admin/`	Django admin URL namespace	GET / POST	Django staff/superuser authorization; application director role alone does not grant admin access.
`/accounts/login/`	`login`	GET, POST	Public login
`/accounts/register/`	`register`	GET, POST	Public student registration
`/accounts/logout/`	`logout`	POST	Authenticated
`/accounts/password/change/`	`password_change`	GET, POST	Authenticated
`/accounts/password/change/done/`	`password_change_done`	GET	Authenticated

Duplicate paths and aliases

Django resolves the first matching URL pattern. Several retained names reverse to the same path and use the same upgraded view:

- `/announcements/` : `announcements` , `announcement` .
- `/complaints/` : `complaints` , `complaints_list` .

The legacy `/score/<submission_id>/` path looks up a Submission ID; the modern quiz result route uses a Quiz ID. Path case matters: `/Director_dashboard/` retains its original capitalization.

Query parameters

Page	Parameter	Behavior
Resources	q	Case-insensitive title, description and subject search; stripped to 150 characters.
Resources	category	notes, pyq or assignment; matches the stored category.
Resources	subject	Case-insensitive exact subject; stripped to 100 characters.
Resources	sort=title	Alphabetical title, then ID; otherwise model ordering is newest first.
Quizzes	code	Uppercase exact quiz code, stripped to six characters.
Complaints	status	Stored status code; pending, in_progress or resolved.
Account creation	role	GET selects faculty, student or director; faculty is the default. Legacy staff routes fix the role.
Paginated lists	page	Page number; 15 records per page.

Configuration reference

Updated 21 October 2026. Values below are placeholders, not live credentials.

Loading and precedence

Classroom/settings.py loads .env without overriding process variables. WSGI, ASGI and management commands therefore use the process environment first. The deliberate exception is local manage.py runserver: it loads the project's .env with override=True before Django starts. Do not use that behavior to infer Vercel values; configure Vercel environment variables separately.

Booleans accept 1, true and yes, case-insensitively. Other strings, including release, are false. Comma-separated host/origin lists are trimmed.

Environment variable index

Variable	Default / use
DEBUG	True locally, false when VERCEL is present; set False explicitly in production
SECRET_KEY	Locally reads/creates .dev-secret if blank; production requires at least 50 characters
DATABASE_URL	Blank selects local SQLite; otherwise dj-database-url parses the URI
ALLOWED_HOSTS	localhost,127.0.0.1,[::1]; add explicit custom production domains without schemes
CSRF_TRUSTED_ORIGINS	Explicit additional origins, including https://
VERCEL	Presence changes DEBUG's fallback to false
VERCEL_URL	Provided deployment hostname; appended to host and HTTPS origin lists
VERCEL_PROJECT_PRODUCTION_URL	Provided production hostname; appended to host/origin lists
VERCEL_BRANCH_URL	Provided branch hostname; appended to host/origin lists
GS_BUCKET_NAME	Optional private Google Cloud Storage bucket; see upload modes below
TRUST_PROXY_HEADERS	False; True accepts HTTP_X_FORWARDED_PROTO=https from a trusted proxy
DEMO_PASSWORD	Explicit chosen password for development seed_demo; never a default production password
PORT	Container bind port, default 8080; not read by the Django settings parser
DJANGO_SETTINGS_MODULE	Classroom.settings in standard entry points

EMAIL_* and DEFAULT_FROM_EMAIL environment variables are no longer read. Outbound email uses django.core.mail.backends.dummy.EmailBackend. Public reset links, email/phone OTP and SMTP setup are removed. Signed-in password change remains.

Database backends

Backend	URI shape	Behavior
SQLite	sqlite:///PATH	Local development/test fallback
PostgreSQL	postgresql://USER:PASSWORD@HOST:PORT/DATABASE	psycopg from main requirements

Backend	URI shape	Behavior
MySQL / Aiven	mysql://USER:URL_ENCODED_PASSWORD@HOST:PORT/DATABASE	mysql-connector-python from main requirements

MySQL configuration maps django.db.backends.mysql to mysql.connector.django. OPTIONS contains use_pure=True, ssl_ca pointing at certs/aiven-ca.pem, ssl_verify_cert=True and ssl_verify_identity=True. The public CA is committed; private credentials and other private PEM/key files remain excluded. Use a certificate matching the service and an existing database/user.

Connections use conn_max_age=60 and conn_health_checks=True. This is configuration, not a connection or schema-readiness test. Never copy an unencoded password into an example URI when it contains reserved URL characters.

Upload modes

Mode	Default storage	Uploads
DEBUG=True, no bucket	Local MEDIA_ROOT	Enabled for development
GS_BUCKET_NAME configured	Private GCS backend	Enabled; runtime credentials/permissions still required
DEBUG=False, no bucket	ClassApp.storage.DisabledUploadStorage	Disabled

UPLOADS_ENABLED is derived from DEBUG or the bucket name. The form explains the unavailable state, middleware rejects file-bearing mutation requests, and the storage backend refuses writes. Text-only workflows remain available. A bucket is not required for production startup. Supabase is not implemented.

Fixed settings and limits

- Production requires a nonempty SECRET_KEY of at least 50 characters and either

PostgreSQL or mysql.connector.django. SQLite is rejected in production.

- TIME_ZONE=Asia/Kolkata; USE_TZ=True. Quiz launch/deadline and submission times

are shown in IST. A batch is descriptive text, not a timetable assignment.

- STORAGES.staticfiles uses CompressedManifestStaticFilesStorage; run collectstatic

for production, including theme.js, theme.css and favicon.svg.

- Session age is 3600 seconds; browser-close expiry, HTTP-only cookies and

SameSite=Lax are configured. DEBUG=False enables secure cookies, HTTPS redirect and HSTS. Only trust forwarded HTTPS headers behind a trusted proxy.

- Password validation requires at least 10 characters plus Django's similarity,

common-password and numeric-password checks.

- Files are limited to 10 MB by form validation; declared request bodies above

12 MB are rejected. File content/image validation and private download authorization remain in place when storage is configured.

- Lists use 15 records per page. Quiz builders allow 1-50 questions. Login failure

thresholds are 10 attempts per 15-minute username/peer-IP window.

Examples

Development .env should be private. Copy .env.example and fill actual values. For isolated checks, use a separate terminal with DEBUG=True and DATABASE_URL=sqlite:///:memory:; close that terminal before normal database work. See Vercel deployment for the current Aiven/Vercel example and Google Cloud guide for the optional container/GCS deployment.

`manage.py check` validates Django configuration. `check --deploy` evaluates production settings. Neither proves live database access, schema status, storage permissions, deployed commit identity or DNS health. /health/ checks the process, not database readiness. Secrets belong in private .env or platform secret storage.

Complete colour index

Generated from `app.css` and `theme.css` on 21 October 2026: 135 distinct hex literals plus named CSS colours. This is an inventory of the active application stylesheet, not of photographic pixels, browser defaults or the retained legacy theme.

Theme variables

Token	Declared value
`:root / --bg`	`#f5f7fa`
`:root / --ink`	`#172d35`
`:root / --muted`	`#6e7e86`
`:root / --line`	`#e5ebee`
`:root / --teal`	`#126c64`
`:root / --teal-dark`	`#0e514b`
`:root / --pale`	`#e8f4ef`
`:root / --white`	`#fff`
`:root / --shadow`	`0 3px 12px #19353c04`
`:root / --muted`	`#596b75`
`[data-theme=dark] / --bg`	`#10191f`
`[data-theme=dark] / --ink`	`#e6edf1`
`[data-theme=dark] / --muted`	`#a5b5be`
`[data-theme=dark] / --line`	`#33444e`
`[data-theme=dark] / --teal`	`#76d2bc`
`[data-theme=dark] / --teal-dark`	`#247665`
`[data-theme=dark] / --pale`	`#203d36`
`[data-theme=dark] / --white`	`#1a2831`
`[data-theme=dark] / --shadow`	`0 3px 12px #0002`

Every explicit colour and its source usage

Three-digit hex expands each digit twice. Eight-digit hex is RRGGBBAA, with the last byte specifying alpha. RGB below is the uncomposited colour; its visible appearance also depends on opacity and the underlying surface. Named white aliases #ffffff, while transparent has zero opacity. `currentColor` in SVG arrows inherits its surrounding text colour.

CSS literal	RGB	Opacity	Selectors and properties
`#0002`	0, 0, 0	13.33%	`.theme-toggle → box-shadow`; `[data-theme=dark] → --shadow`
`#0e514b`	14, 81, 75	100.00%	`:root → --teal-dark`

CSS literal	RGB	Opacity	Selectors and properties
`#10191f`	16, 25, 31	100.00%	`[data-theme=dark] → --bg`
`#101e19`	16, 30, 25	100.00%	`[data-theme=dark] .site-footer → background`
`#102b24e8`	16, 43, 36	90.98%	`.campus-slide figcaption → background`
`#111e26`	17, 30, 38	100.00%	`[data-theme=dark] :is(input:not([type=checkbox]):not([type=radio]), select, textarea) → background`
`#122d2526`	18, 45, 37	14.90%	`.carousel-arrow → box-shadow`
`#126c64`	18, 108, 100	100.00%	`:root → --teal`
`#143c35`	20, 60, 53	100.00%	`.site-footer → background`
`#156c62`	21, 108, 98	100.00%	`.art-small → background`
`#172d35`	23, 45, 53	100.00%	`:root → --ink`
`#17303322`	23, 48, 51	13.33%	`.sidebar → box-shadow`
`#173e3512`	23, 62, 53	7.06%	`.carousel-stage → box-shadow`
`#176c5c`	23, 108, 92	100.00%	`[data-theme=dark] :is(button, .button):not(.secondary):not(.theme-toggle) → background`
`#19353c04`	25, 53, 60	1.57%	`:root → --shadow`
`#193e35`	25, 62, 53	100.00%	`.campus-slide-portrait img → background` ; `.carousel-stage → background`
`#1a2831`	26, 40, 49	100.00%	`[data-theme=dark] → --white`
`#1c493214`	28, 73, 50	7.84%	`.campus-cover → box-shadow`
`#203d32`	32, 61, 50	100.00%	`[data-theme=dark] :is(.notification.unread, .alert) → background`
`#203d36`	32, 61, 54	100.00%	`[data-theme=dark] → --pale`
`#204c3e`	32, 76, 62	100.00%	`.site-footer .brand-icon → color`
`#247665`	36, 118, 101	100.00%	`[data-theme=dark] → --teal-dark`
`#283d37`	40, 61, 55	100.00%	`[data-theme=dark] .status → background`
`#33444e`	51, 68, 78	100.00%	`[data-theme=dark] → --line`
`#338b78`	51, 139, 120	100.00%	`[data-theme=dark] :is(button, .button):not(.secondary):not(.theme-toggle) → border-color`
`#36744f`	54, 116, 79	100.00%	`.alert → color`
`#3c8261`	60, 130, 97	100.00%	`.resolved → color`
`#3c8d70`	60, 141, 112	100.00%	`.file-icon → color`

CSS literal	RGB	Opacity	Selectors and properties
`#406452`	64, 100, 82	100.00%	`[data-theme=dark] :is(.notification.unread, .alert) → border-color`
`#432827`	67, 40, 39	100.00%	`[data-theme=dark] :is(.alert.error, .errorlist) → background`
`#435f56`	67, 95, 86	100.00%	`.home-explore → color`
`#443b24`	68, 59, 36	100.00%	`[data-theme=dark] .status.pending → background`
`#46614f`	70, 97, 79	100.00%	`.campus-photo-grid figcaption → color`
`#4b6255`	75, 98, 85	100.00%	`.campus-identity strong → color`
`#539b84`	83, 155, 132	100.00%	`progress::-moz-progress-bar → background` ; `progress::-webkit-progress-value → background`
`#5474ae`	84, 116, 174	100.00%	`.in_progress → color`
`#596b75`	89, 107, 117	100.00%	`:root → --muted`
`#5b9d84`	91, 157, 132	100.00%	`.timeline article:before → background`
`#5c7566`	92, 117, 102	100.00%	`.campus-visit p → color`
`#62776a`	98, 119, 106	100.00%	`.campus-facilities p → color`
`#63766f`	99, 118, 111	100.00%	`.site-nav a → color`
`#647c6c`	100, 124, 108	100.00%	`.campus-cover figcaption → color`
`#65a98d`	101, 169, 141	100.00%	`.notification.unread → border-left`
`#668179`	102, 129, 121	100.00%	`.welcome-banner p → color`
`#687b81`	104, 123, 129	100.00%	`.sidebar nav a → color`
`#6a7b82`	106, 123, 130	100.00%	`.top-actions a → color`
`#6b7e73`	107, 126, 115	100.00%	`.carousel-heading → color` ; `.carousel-position → color`
`#6b7e85`	107, 126, 133	100.00%	`.stat-card>span → color`
`#6e7e86`	110, 126, 134	100.00%	`:root → --muted`
`#708475`	112, 132, 117	100.00%	`.campus-location-tag → color`
`#75858c`	117, 133, 140	100.00%	`.announcement-preview p → color`
`#75858d`	117, 133, 141	100.00%	`.empty → color`
`#76a88b`	118, 168, 139	100.00%	`.campus-marker → background`
`#76d2bc`	118, 210, 188	100.00%	`[data-theme=dark] → --teal`
`#784640`	120, 70, 64	100.00%	`[data-theme=dark] :is(.alert.error, .errorlist) → border-color`

CSS literal	RGB	Opacity	Selectors and properties
`#7b8a81`	123, 138, 129	100.00%	`.campus-breadcrumb → color`
`#7c8a80`	124, 138, 128	100.00%	`.campus-source → color`
`#8a9d91`	138, 157, 145	100.00%	`.feature-number → color`
`#94a0a5`	148, 160, 165	100.00%	`.workspace → color`
`#99a6aa`	153, 166, 170	100.00%	`footer → color`
`#9a6b17`	154, 107, 23	100.00%	`.pending → color`
`#a5b5be`	165, 181, 190	100.00%	`[data-theme=dark] → --muted`
`#a5c0af`	165, 192, 175	100.00%	`.footer-about>span → color`
`#a7523e`	167, 82, 62	100.00%	`.alert.error → color`
`#a7c1b0`	167, 193, 176	100.00%	`.footer-bottom → color`
`#a9c9b5`	169, 201, 181	100.00%	`.site-footer .brand-dot → color`
`#ac4239`	172, 66, 57	100.00%	`.errorlist → color`
`#b0e4cc`	176, 228, 204	100.00%	`[data-theme=dark] .status → color`
`#b8efd0`	184, 239, 208	100.00%	`[data-theme=dark] :is(.notification.unread, .alert) → color`
`#bd8c3a`	189, 140, 58	100.00%	`.list-row:nth-child(3n) .file-icon → color`
`#c2d5c9`	194, 213, 201	100.00%	`.footer-about p → color`
`#c4d8cb`	196, 216, 203	100.00%	`.site-footer nav a → color`
`#c6d4ca`	198, 212, 202	100.00%	`.carousel-dots button::after → background`
`#cadbd0`	202, 219, 208	100.00%	`.header-sign-in → border`
`#cbe5d2`	203, 229, 210	100.00%	`.alert → border`
`#cfe3d7`	207, 227, 215	100.00%	`.art-square → box-shadow`
`#d3e0d9`	211, 224, 217	100.00%	`.site-footer → color`
`#d5e4da`	213, 228, 218	100.00%	`.footer-bottom a → color`
`#d6e1e3`	214, 225, 227	100.00%	`input:not([type=checkbox]):not([type=radio]),select,textarea → border`
`#d7e7dc`	215, 231, 220	100.00%	`.site-footer .brand-icon → background`
`#d8e2e4`	216, 226, 228	100.00%	`.secondary → border-color`
`#d9e7dc`	217, 231, 220	100.00%	`.campus-visit → border`
`#d9e9e0`	217, 233, 224	100.00%	`.welcome-banner → border`
`#dbece4`	219, 236, 228	100.00%	`.timeline → border-left`
`#dce8e0`	220, 232, 224	100.00%	`.auth-tiles>div → border`
`#dceee5`	220, 238, 229	100.00%	`.avatar → background`

CSS literal	RGB	Opacity	Selectors and properties
`#dfe7e1`	223, 231, 225	100.00%	`.campus-facilities article → border` ; `.campus-photo-grid figure → border` ; `.campus-space-card → border`
`#dfe8e2`	223, 232, 226	100.00%	`.campus-location-tag → border-top` ; `.campus-spaces → border-top` ; `.home-features → border-top`
`#dfe9e1`	223, 233, 225	100.00%	`.campus-photo-grid .campus-photo-portrait img → background`
`#dfece3`	223, 236, 227	100.00%	`.campus-facilities .file-icon → background`
`#e0e8e2`	224, 232, 226	100.00%	`.home-features .panel → border-color`
`#e0eae4`	224, 234, 228	100.00%	`.campus-slide figcaption p → color`
`#e1ece5`	225, 236, 229	100.00%	`.site-footer h2 → color`
`#e1f1e9`	225, 241, 233	100.00%	`.badge → background`
`#e2e8e2`	226, 232, 226	100.00%	`.campus-identity img → border`
`#e2e9e4`	226, 233, 228	100.00%	`.site-header → border-bottom`
`#e3eee7`	227, 238, 231	100.00%	`.campus-slide figcaption>span → color`
`#e5ebee`	229, 235, 238	100.00%	`:root → --line`
`#e5eee7`	229, 238, 231	100.00%	`.carousel-dots button:hover → background`
`#e5eeea`	229, 238, 234	100.00%	`.tag → border`
`#e5efe7`	229, 239, 231	100.00%	`.campus-marker → box-shadow`
`#e5f0e9`	229, 240, 233	100.00%	`.auth-tiles>div → background`
`#e6edf1`	230, 237, 241	100.00%	`[data-theme=dark] → --ink`
`#e6f0e8`	230, 240, 232	100.00%	`.campus-visit → background`
`#e7af49`	231, 175, 73	100.00%	`button:focus-visible,a:focus-visible,input:focus-visible,textarea:focus-visible,select:focus-visible → outline`
`#e7f2ec`	231, 242, 236	100.00%	`.welcome-banner → background`
`#e7f4ec`	231, 244, 236	100.00%	`.resolved → background`
`#e8f4eb`	232, 244, 235	100.00%	`.alert → background`
`#e8f4ef`	232, 244, 239	100.00%	`:root → --pale`
`#e9b55c`	233, 181, 92	100.00%	`.art-circle → color`
`#eaf0ff`	234, 240, 255	100.00%	`.in_progress → background`
`#edf3ee`	237, 243, 238	100.00%	`.campus-facilities article → background`

CSS literal	RGB	Opacity	Selectors and properties
`#edf3ef`	237, 243, 239	100.00%	`progress → background` ; `progress::-webkit-progress-bar → background`
`#eef5f1`	238, 245, 241	100.00%	`.file-icon → background`
`#efd5ce`	239, 213, 206	100.00%	`.alert.error → border-color`
`#f0f3f4`	240, 243, 244	100.00%	`.announcement-preview → border-top` ; `.list-row → border-top` ; `td → border-bottom`
`#f0f5f3`	240, 245, 243	100.00%	`.tag → background`
`#f0f7f3`	240, 247, 243	100.00%	`.answer-box → background`
`#f0f8f3`	240, 248, 243	100.00%	`.notification.unread → background`
`#f4d48d`	244, 212, 141	100.00%	`[data-theme=dark] .status.pending → color`
`#f4f6f6`	244, 246, 246	100.00%	`code → background`
`#f5f7fa`	245, 247, 250	100.00%	`:root → --bg`
`#f7f9f7`	247, 249, 247	100.00%	`.public-site → background`
`#fafcfb`	250, 252, 251	100.00%	`.status-grid>div → background`
`#ffc2ba`	255, 194, 186	100.00%	`[data-theme=dark] :is(.alert.error, .errorlist) → color`
`#fff`	255, 255, 255	100.00%	`.campus-cover → background` ; `.campus-cover → border` ; `.campus-identity img → background` ; `.campus-photo-grid figure → background` ; `.campus-slide figcaption → color` ; `.campus-space-card → background` ; `.carousel-arrow:hover → border-color` ; `.home-features .panel → background` ; `.light → background` ; `.light → border-color` ; `.role-switch a[aria-current=page] → color` ; `.site-footer .brand → color` ; `.site-footer a:hover → color` ; `.skip-link → color` ; `:root → --white` ; `[data-theme=dark] :is(button, .button):not(.secondary):not(.theme-toggle) → color` ; `input:not([type=checkbox]):not([type=radio]),select,textarea → background`
`#fff0ed`	255, 240, 237	100.00%	`.errorlist → background`
`#fff1ed`	255, 241, 237	100.00%	`.alert.error → background`
`#fff3dc`	255, 243, 220	100.00%	`.pending → background`
`#fff5e6`	255, 245, 230	100.00%	`.list-row:nth-child(3n) .file-icon → background`
`#ffffff20`	255, 255, 255	12.55%	`.footer-bottom → border-top`

CSS literal	RGB	Opacity	Selectors and properties
`#ffffff80`	255, 255, 255	50.20%	`.carousel-arrow → border`
`#ffffffc9`	255, 255, 255	78.82%	`.topbar → background`
`#fffffff2`	255, 255, 255	94.90%	`.carousel-arrow → background`
`#fffffff5`	255, 255, 255	96.08%	`.site-header → background`
`transparent`	0, 0, 0	0.00%	`.campus-slide figcaption → background` ; `.carousel-dots button → background`
`white`	255, 255, 255	100.00%	`.art-small → color` ; `.art-square → background` ; `.brand-icon → color` ; `.panel → background` ; `.sidebar → background` ; `.stat-card → background` ; `.timeline article:before → border` ; `button,.button → color` ; `button:hover,.button:hover → color`

Test catalogue

57 test methods are defined in `ClassApp/tests.py`. This catalogue describes source coverage; a successful test execution must be recorded separately.

Test class	Method	Scenario
WorkspaceTests	`test_all_role_pages_render`	all role pages render
WorkspaceTests	`test_guests_require_login`	guests require login
WorkspaceTests	`test_student_cannot_access_staff_actions`	student cannot access staff actions
WorkspaceTests	`test_registration_hashes_password_and_ignores_role_injection`	registration hashes password and ignores role injection
WorkspaceTests	`test_login_throttle`	login throttle
WorkspaceTests	`test_role_login_pages_and_matching_accounts`	role login pages and matching accounts
WorkspaceTests	`test_role_login_mismatch_explains_where_to_sign_in`	role login mismatch explains where to sign in
WorkspaceTests	`test_csrf_and_logout_method`	csrf and logout method
WorkspaceTests	`test_email_password_reset_is_removed`	email password reset is removed
WorkspaceTests	`test_resource_upload_search_and_notification`	resource upload search and notification
WorkspaceTests	`test_reject_fake_pdf_and_html`	reject fake pdf and html
WorkspaceTests	`test_oversized_and_binary_txt_rejected`	oversized and binary txt rejected
WorkspaceTests	`test_download_authenticated_and_counted`	download authenticated and counted
WorkspaceTests	`test_missing_resource_does_not_increment`	missing resource does not increment
WorkspaceTests	`test_notifications_are_private_and_mutations_post_only`	notifications are private and mutations post only
WorkspaceTests	`test_notification_open_redirect_blocked`	notification open redirect blocked
WorkspaceTests	`test_announcement_audience`	announcement audience
WorkspaceTests	`test_create_quiz_formset`	create quiz formset
WorkspaceTests	`test_quiz_rejects_empty_questions`	quiz rejects empty questions
WorkspaceTests	`test_quiz_scoring_and_duplicate_submission`	quiz scoring and duplicate submission
WorkspaceTests	`test_answers_withheld_until_close`	answers withheld until close
WorkspaceTests	`test_incomplete_quiz_is_not_submitted`	incomplete quiz is not submitted
WorkspaceTests	`test_closed_upcoming_and_unpublished_quizzes`	closed upcoming and unpublished quizzes
WorkspaceTests	`test_other_faculty_cannot_manage_quiz`	other faculty cannot manage quiz
WorkspaceTests	`test_students_only_see_their_submission_details`	students only see their submission details
WorkspaceTests	`test_complaint_creation_and_status_history`	complaint creation and status history
WorkspaceTests	`test_complaint_object_access`	complaint object access
WorkspaceTests	`test_feedback_upsert_and_range`	feedback upsert and range

Test class	Method	Scenario
WorkspaceTests	`test_feedback_db_constraint`	feedback db constraint
WorkspaceTests	`test_doubt_answer_permission_and_notification`	doubt answer permission and notification
WorkspaceTests	`test_account_creation_and_deactivation`	account creation and deactivation
WorkspaceTests	`test_profile_edit_for_all_roles`	profile edit for all roles
WorkspaceTests	`test_profile_edit_rejects_duplicate_email_and_username`	profile edit rejects duplicate email and username
WorkspaceTests	`test_profile_edit_requires_login_and_csrf`	profile edit requires login and csrf
WorkspaceTests	`test_staff_creation_forms_exclude_student_details`	staff creation forms exclude student details
WorkspaceTests	`test_staff_registration_fixes_role_and_saves_contact_details`	staff registration fixes role and saves contact details
WorkspaceTests	`test_director_created_student_requires_and_saves_academic_details`	director created student requires and saves academic details
WorkspaceTests	`test_director_cannot_deactivate_self`	director cannot deactivate self
WorkspaceTests	`test_report_export_alphabetical_and_csv_safe`	report export alphabetical and csv safe
WorkspaceTests	`test_dashboard_no_removed_performance_sections`	dashboard no removed performance sections
WorkspaceTests	`test_legacy_import_dry_run_and_hashing`	legacy import dry run and hashing
WorkspaceTests	`test_director_bootstrap_command`	director bootstrap command
WorkspaceTests	`test_password_change_invalidates_old_password`	password change invalidates old password
WorkspaceTests	`test_deactivation_revokes_existing_session`	deactivation revokes existing session
WorkspaceTests	`test_original_role_urls_enforce_correct_role`	original role urls enforce correct role
WorkspaceTests	`test_original_quiz_answer_and_detail_tables_are_written`	original quiz answer and detail tables are written
WorkspaceTests	`test_multi_select_requires_exact_correct_set`	multi select requires exact correct set
WorkspaceTests	`test_forged_options_from_other_question_rejected`	forged options from other question rejected
WorkspaceTests	`test_required_original_quiz_details_are_validated`	required original quiz details are validated
WorkspaceTests	`test_quiz_attempt_autofills_separate_academic_details_and_dates`	quiz attempt autofills separate academic details and dates
WorkspaceTests	`test_invalid_legacy_quiz_cannot_be_attempted_or_published`	invalid legacy quiz cannot be attempted or published
WorkspaceTests	`test_private_original_doubt_attachment`	private original doubt attachment
WorkspaceTests	`test_original_announcement_upload_and_audience_download`	original announcement upload and audience download
WorkspaceTests	`test_score_url_checks_original_submission_id`	score url checks original submission id
WorkspaceTests	`test_student_academic_fields_are_preserved`	student academic fields are preserved
WorkspaceTests	`test_plaintext_fields_no_longer_exist`	plaintext fields no longer exist

Test class	Method	Scenario
LegacyMigrationTests	`test_populated_original_0006_database_upgrades_without_losing_records`	populated original 0006 database upgrades without losing records

Vercel and Aiven deployment

Deploy the latest intended Git commit; retrying an older deployment can continue to serve old templates and assets. Inspect the commit on the active deployment, then use that deployment's Visit URL. A successful GitHub push alone does not prove which version a browser is viewing.

Environment

Configure these in Vercel for every environment that should run the application:

```
DEBUG=False
SECRET_KEY=YOUR_PRIVATE_RANDOM_KEY_AT_LEAST_50_CHARACTERS
DATABASE_URL=mysql://USER:URL_ENCODED_PASSWORD@YOUR_AIVEN_HOST:PORT/DATABASE
ALLOWED_HOSTS=YOUR_CUSTOM_OR_STABLE_HOSTNAME
CSRF_TRUSTED_ORIGINS=https://YOUR_CUSTOM_OR_STABLE_HOSTNAME
```

Use the actual Aiven database name and credentials. Copy only each value into Vercel's value field, without the KEY= prefix or quotes. Local .env files are ignored by Git and are not transferred to Vercel. VERCEL_URL, VERCEL_PROJECT_PRODUCTION_URL and VERCEL_BRANCH_URL are recognized automatically when provided, accepting those exact hosts and HTTPS origins. DEBUG defaults off when VERCEL is present; explicitly set it false for deployment.

The main requirements include mysql-connector-python. The MySQL engine parsed by dj-database-url is remapped to mysql.connector.django. TLS uses use_pure=True, ssl_ca, ssl_verify_cert=True and ssl_verify_identity=True. The public certificate at certs/aiven-ca.pem must match the service. The Git exception includes that CA certificate only; private keys and passwords must remain excluded.

Schema and startup

Run migrations through a trusted terminal configured for the intended database:

```
.\.venv\Scripts\python.exe manage.py migrate
.\.venv\Scripts\python.exe manage.py check --deploy
```

Migration 0010 adds batch and separate submission academic snapshots. Preserve existing records and back up populated databases before upgrades. Migrations are not run on every web request. See UPGRADE.md for original-schema migrations.

Files and email

GS_BUCKET_NAME can be blank. In production, uploads then remain disabled and show a clear message; do not rely on Vercel's local filesystem to preserve uploads. To enable files, provision private GCS storage and its credentials first. Aiven stores application records, not the uploaded file bodies. Supabase storage is not integrated.

Public password reset, email/SMS OTP and SMTP sending are removed. EMAIL_* variables are unused and can be removed from Vercel. If an App Password was created solely for this retired feature, revoke it in the provider account. Signed-in users can change passwords through the Password link.

Post-deployment checks

1. Confirm the intended commit is Ready and open its Visit URL.

2. Verify the homepage, Contact header/footer, login and mobile navigation.
3. Verify the bottom-right theme toggle and saved choice after navigation.
4. Check that theme.js, theme.css, favicon.svg and application CSS return actual

assets, not HTML login/error pages. Production asset filenames can be hashed.

1. Sign in as each role. Test director faculty creation, own profile edits and

the student quiz form/result layout with an appropriate test account.

1. Read the actual failing request's runtime traceback if a 500 occurs; a

successful homepage request does not establish database-backed flows work.

Deployment protection can redirect an independent visitor to Vercel login. That redirect prevents unauthenticated live inspection; inspect the permitted deployment in your account or provide an intentionally public production URL.

Current features and recent changes

Updated 21 October 2026. This describes the current repository, not proof that an older Vercel deployment serves the same commit.

Accounts and profiles

- Students, faculty and directors open My profile by clicking their name/role in

the sidebar footer. The route is `/accounts/profile/`.

- Name, username, email, department and phone are editable for the signed-in user.

Students additionally edit enrollment, year, semester, section and batch.

- Updates synchronize the linked Student/Faculty/Director and complaint-wrapper

names. Role, administrator flags and another user's identity cannot be edited through this form. Duplicate usernames/emails produce validation errors.

- Email/SMS OTP, Forgot password pages, public reset links, Gmail SMTP settings

and the temporary check_email command have been removed. Outbound email uses Django's dummy backend. Signed-in password change still requires the old password.

Quizzes

- Creation shows title, subject, launch time, deadline and the question builder.

Publication and the four identity requirements are applied automatically. Publication can still be changed by the existing authorized toggle.

- Student attempts autofill name, enrollment and email from the account. Course,

year, semester, section and batch have separate inputs, with existing values prefilled. Missing details can be completed before submission.

- Launch date/time and submission deadline are labelled explicitly in IST.
- Question numbers sit inside each card above the question. Panels and long

options wrap within the form width.

- Results show score, an IST submission timestamp and an academic-detail grid

that adapts to smaller screens. Answers remain hidden from students until close.

- Migration 0010 adds Student.batch and Submission.academic_details. The JSON

snapshot preserves separate values; the older class_div field remains as a compatibility summary. Old attempts are not backfilled with guessed data.

Shared website behavior

- Sun/moon button at bottom-right on public, login and dashboard pages. The choice

persists in browser local storage, syncs across tabs, and initially follows the device theme. Restricted local storage does not break the current page.

- Contact uses the public header/footer and navigation. Messages are saved in

Contact and visible to authorized Django administrators, not role dashboards; submitting the form does not send email.

- Keyboard skip link, visible focus, reduced-motion support, clearer text contrast,

mobile controls and explicit form actions are shared by current templates.

- Form navigation shows progress and prevents duplicate submissions. Image decoding

is asynchronous; secondary campus photos retain lazy loading and image dimensions.

Production and storage

- Aiven MySQL uses mysql.connector.django and the committed public CA certificate;

credentials remain private. PostgreSQL and local SQLite remain supported.

- Vercel-provided deployment, production and branch hostnames are added to host

and CSRF-origin lists. No global wildcard is required.

- A GCS bucket is optional for startup. In production without a bucket, file

uploads are disabled at the form, request and storage layers; text workflows remain available. There is no Supabase integration yet.

- Local `manage.py runserver` loads .env with override enabled. Other commands and

deployed WSGI/ASGI use normal process-environment precedence.

Verification record

The latest feature-removal run passed 57 Django tests. Earlier local Edge checks covered public pages at 390/768/1440 pixels, role navigation, faculty creation, quiz autofill/submission/results, keyboard access and actual dark colours. Production checks, static collection and dependency checks passed during the quality review. These observations do not certify a protected live Vercel site, all browser engines, external content, production load or a bug-free application.

Testing and verification

Run `python manage.py test ClassApp` from the project folder. Tests use a separate temporary database and in-memory media storage. They do not edit the original supplied database.

The current source defines **57 test methods**, listed individually in TEST_CATALOGUE.md. Use the same virtual-environment interpreter for Django and its test subprocesses. For a workspace with an existing MySQL database, prefer the isolated SQLite procedure in handbook section 15; it preserves the shell's previous configuration after the run.

Current verification record

The feature-removal run passed all 57 tests, including unavailable reset URLs and retained signed-in password changes. Coverage now includes per-role profile edits, role/permission injection rejection, separate quiz autofill/snapshots and automatic quiz defaults. Local Edge flows checked 390/768/1440px public pages, role navigation, faculty creation, quiz submission/results, keyboard access and dark-mode persistence. Safari/Firefox and protected live Vercel pages were not independently verified.

Documentation-only changes do not require a new application test run. Rebuild, PDF-export and link checks use the commands in tools/README.md.

Automated coverage

The suite covers:

- All role dashboards and module pages; anonymous redirects; denial of staff actions to students; correct protection on original role URLs.
- Registration password hashing, prevention of role injection, retained student academic fields and director account bootstrap.
- Login throttling/cooldown, CSRF, POST-only logout, password changes and removed reset routes, account deactivation and revocation of existing sessions.
- Resource upload/search/filters, file content/size validation, authenticated downloads, counting, unavailable files and no public media route.
- Notification privacy, mark-read actions, unsafe redirect rejection and audience restrictions.
- Original announcement document upload, author/faculty links and authorized document download.
- Quiz creation/minimum questions, opening/closing/publication controls, required student details, strict option membership and single/multi-select scoring.
- One authenticated attempt, preserved original answer/detail tables, student/faculty ownership, delayed solutions, invalid legacy quiz rejection and original score URL access.
- Complaints, assignment, role-controlled status/history, doubts and assigned replies, private legacy attachments.
- Feedback updates and database rating constraints; alphabetical CSV exports and formula-text protection.
- Legacy student importer dry-run, password hashing, source preservation and conflict handling.
- No plaintext pw fields in current role models.

Populated original-schema migration test

An isolated database is first migrated to the original 0006 schema and populated with legacy students, faculty, directors, existing Django/complaint accounts, a username conflict, quiz questions/images/options, repeated submissions with answer/detail M2M relations, repeated ratings, a doubt attachment, an announcement document and a replied complaint.

It then applies 0007–0009 and verifies:

- All original record IDs and relationships remain.
- Legacy plaintext passwords are converted; recognized hashes remain usable; old Django users are not silently taken over by a conflicting legacy username.
- Profiles and role login work.
- Question images, answer selections, submission details and attachment paths remain intact.
- Ambiguous/repeated historical records remain stored; one newest record receives each unique authenticated link.
- Complaint text, teacher reply, status and timeline are retained.
- Old sessions and OTPs are cleared; legacy plaintext password columns no longer exist.

This validates the Django migration path on SQLite. It does not claim validation against an inaccessible external MySQL server.

Browser checks

Chromium checks all three role dashboards at desktop (1440 px) and mobile (390 px), role module navigation, mobile sidebar behavior, resource download, student quiz submission and the dynamic question-builder button. Rendered pages were inspected for overflow and JavaScript errors.

Recent public-page checks used the running local HTTP server in a hidden Chrome preview. The homepage has seven carousel slides/dots; both newly added photos load, arrows wrap between first/last, keyboard and touch controls work, and the removed star/tagline block is absent. The Campus Life header/footer links open a dedicated page, its selected main image is displayed in full, and all 13 current image elements have valid local asset responses. Narrow layouts were checked at 320 and 390 px, with tablet/desktop checks during the public-page changes. These local observations do not replace browser acceptance in a live deployment.

Fresh package checks to repeat for a release

Extract the release into a new directory, create its environment without overwriting an existing configuration, then check migrations, system/migration drift, demo seeding, static collection and all three demo-role logins against a new isolated database. A production `check --deploy --fail-level WARNING` using synthetic values checks configuration only; it does not establish cloud connectivity. Record the commands, release source, interpreter, database engine and result for each release instead of treating a previous package check as current evidence.

Documentation checks

Run `.venv\Scripts\python.exe tools/build_documentation.py` to rebuild source-derived model, form, route, colour and test references and the offline HTML handbook. Check local document links, handbook anchors, diagram rendering and the visual colour filter. The generator reads metadata/source without querying live user data. It does not execute the application test suite or automatically refresh the PDF snapshot.

The most recent application-suite execution recorded during the current workspace changes passed all 57 tests. Documentation-only changes should be checked through document generation and reference/link validation; they do not require claiming a fresh application-test execution. Windows compatibility handling includes closing temporary SQLite source handles and launching migration subprocesses with the test environment's Python interpreter.

Manual acceptance and live production checks

1. Create a director, faculty and student; test original login/dashboard links and the new home page.
2. Upload resource categories, test search/filters and download counts.
3. Publish a quiz with a single-choice question, multi-select question and image. Test required student details, exact-set grading, duplicate prevention and the answer-review closing time.

4. Upload an announcement document and a doubt attachment. Confirm that another student's private doubt cannot be opened.
5. Submit/update feedback, reply to doubts and walk a complaint through all three statuses.
6. Review director charts, resource totals, historical records and all four CSV reports.
7. Review the migrated account-name report and any unlinked historical records against your backup.
8. Rehearse the real MySQL database upgrade separately if using that database. Test media path preservation and backups.
9. In Google Cloud, verify Cloud SQL connectivity, private bucket reads/writes, HTTPS cookies, CSRF origins, static assets and the health endpoint.
10. Set ingress body/file-size enforcement, including streamed requests without Content-Length, and load-test your intended student count. Review login throttling behind the actual proxy because peer-IP limits can group several users.

Expected Forbidden/Not Found log entries occur during access-denial tests and are not test failures.

Website quality review — 21 October 2026

This review covers the checked-out source and an isolated local installation. It does not certify the protected Vercel deployment or change Aiven records.

Area	Implementation and verification
Clear purpose	Homepage explains resources, quizzes and campus conversations, with role-based sign-in links. Added a shared page description.
Consistent design	Shared stylesheet, templates, form controls, spacing and persistent light/dark themes. Quiz question numbering and result detail grid are shared components.
Mobile layouts	Real headless Edge checks at 390, 768 and 1440 pixels: public pages had no horizontal overflow. Added mobile form and toolbar improvements.
Loading	Existing lazy-loaded secondary campus images, explicit image dimensions, compressed static storage and deferred application JavaScript. Added asynchronous image decoding. No production latency or load benchmark was performed.
Navigation	Role-based menus, public navigation, profile link, current-page indicators and a keyboard skip link.
Readability	Improved shared muted-text contrast, explicit date labels, individual student academic fields and clear account headings.
Features	All 57 Django tests pass. Browser flows additionally verified role login, module navigation, student quiz submission and director creating a faculty account.
Actions	Existing named action buttons retained. Submission and search buttons now provide progress feedback and guard against duplicate submissions.
Feedback	Existing validation/success/error messages retained. Added busy states and clear unavailable-upload messages before submission.
Accessibility	Skip-to-content works by keyboard, focus outlines exist, theme button has accessible state/name, images have descriptive alternatives, reduced-motion preferences are respected. This is not a complete WCAG certification.
Security	Production Django deployment checks pass. Existing TLS, CSRF, secure cookies, role restrictions, password hashing and Aiven certificate verification retained. Vercel defaults to debug off unless explicitly configured otherwise.
Relevant features	Academic workspace features retained. Uploads remain unavailable in production without persistent storage, with an explanatory message.
Browser compatibility	Headless installed Microsoft Edge verified; responsive CSS and native form controls used. Firefox and Safari were not executed.
Content and links	Local route/render coverage comes from the suite and browser checks. Added favicon. External institutional facts and third-party link availability need periodic owner review.
Maintenance	This record distinguishes verified checks from remaining operational checks. See the commands below and the existing testing/deployment documentation.

Checks before deployment

Use the project virtual environment. For automated tests, explicitly select an isolated database; do not run the suite against the production database.

```
$env:DEBUG = 'True'
$env:DATABASE_URL = 'sqlite:///:memory:'
.\.venv\Scripts\python.exe manage.py test
.\.venv\Scripts\python.exe manage.py makemigrations --check --dry-run
.\.venv\Scripts\python.exe -m pip check
node --check ClassApp/static/ClassApp/app.js
node --check ClassApp/static/ClassApp/theme.js
```

These environment overrides apply to that terminal; close it before resuming normal database work. Browser checks used temporary local accounts and databases, not production credentials. Playwright was installed locally as an audit tool; it is not a runtime dependency.

For a production configuration, also run `manage.py check --deploy` and collect static assets. Confirm Vercel serves the intended commit, and test its actual production URL after deployment. A successful GitHub push alone does not verify the live site.

Remaining operational checks

- Keep `DEBUG=False` on Vercel and use the intended database and allowed domains.
- Check the deployed site's theme assets, login, faculty creation, quiz attempt,

result layout and mobile navigation after publishing.

- Public email password resets and SMTP delivery have been removed. Signed-in

users can still change their password using the Password link.

- Configure persistent private storage before enabling uploads.
- Recheck Safari/Firefox, real mobile devices, external campus links and production

performance separately. Deployment protection prevented independent live inspection.

Visual colour catalogue

Every explicit colour in the active application and light/dark theme stylesheets. Transparent colours are shown over a checkerboard. Search by value, selector or property.

#0002

RGB (0, 0, 0) · 13.33% opacity

.theme-toggle → box-shadow; [data-theme=dark] → --shadow

#0e514b

RGB (14, 81, 75) · 100.00% opacity

:root → --teal-dark

#10191f

RGB (16, 25, 31) · 100.00% opacity

[data-theme=dark] → --bg

#101e19

RGB (16, 30, 25) · 100.00% opacity

[data-theme=dark] .site-footer → background

#102b24e8

RGB (16, 43, 36) · 90.98% opacity

.campus-slide figcaption → background

#111e26

RGB (17, 30, 38) · 100.00% opacity

[data-theme=dark] :is(input:not([type=checkbox]):not([type=radio]), select, textarea) → background

#122d2526

RGB (18, 45, 37) · 14.90% opacity

.carousel-arrow → box-shadow

#126c64

RGB (18, 108, 100) · 100.00% opacity

:root → --teal

#143c35

RGB (20, 60, 53) · 100.00% opacity

.site-footer → background

#156c62

RGB (21, 108, 98) · 100.00% opacity

.art-small → background

#172d35

RGB (23, 45, 53) · 100.00% opacity

:root → --ink

#17303322

RGB (23, 48, 51) · 13.33% opacity

.sidebar → box-shadow

#173e3512

RGB (23, 62, 53) · 7.06% opacity

.carousel-stage → box-shadow

#176c5c

RGB (23, 108, 92) · 100.00% opacity

[data-theme=dark] :is(button, .button):not(.secondary):not(.theme-toggle) → background

#19353c04

RGB (25, 53, 60) · 1.57% opacity

:root → --shadow

#193e35

RGB (25, 62, 53) · 100.00% opacity

.campus-slide-portrait img → background; .carousel-stage → background

#1a2831

RGB (26, 40, 49) · 100.00% opacity

[data-theme=dark] → --white

#1c493214

RGB (28, 73, 50) · 7.84% opacity

.campus-cover → box-shadow

#203d32

RGB (32, 61, 50) · 100.00% opacity

[data-theme=dark] :is(.notification.unread, .alert) → background

#203d36

RGB (32, 61, 54) · 100.00% opacity

[data-theme=dark] → --pale

#204c3e

RGB (32, 76, 62) · 100.00% opacity

.site-footer .brand-icon → color

#247665

RGB (36, 118, 101) · 100.00% opacity

[data-theme=dark] → --teal-dark

#283d37

RGB (40, 61, 55) · 100.00% opacity

[data-theme=dark] .status → background

#33444e

RGB (51, 68, 78) · 100.00% opacity

[data-theme=dark] → --line

#338b78

RGB (51, 139, 120) · 100.00% opacity

[data-theme=dark] :is(button, .button):not(.secondary):not(.theme-toggle) → border-color

#36744f

RGB (54, 116, 79) · 100.00% opacity

.alert → color

#3c8261

RGB (60, 130, 97) · 100.00% opacity

.resolved → color

#3c8d70

RGB (60, 141, 112) · 100.00% opacity

.file-icon → color

#406452

RGB (64, 100, 82) · 100.00% opacity

[data-theme=dark] :is(.notification.unread, .alert) → border-color

#432827

RGB (67, 40, 39) · 100.00% opacity

[data-theme=dark] :is(.alert.error, .errorlist) → background

#435f56

RGB (67, 95, 86) · 100.00% opacity

.home-explore → color

#443b24

RGB (68, 59, 36) · 100.00% opacity

[data-theme=dark] .status.pending → background

#46614f

RGB (70, 97, 79) · 100.00% opacity

.campus-photo-grid figcaption → color

#4b6255

RGB (75, 98, 85) · 100.00% opacity

.campus-identity strong → color

#539b84

RGB (83, 155, 132) · 100.00% opacity

progress::-moz-progress-bar → background; progress::-webkit-progress-value → background

#5474ae

RGB (84, 116, 174) · 100.00% opacity

.in_progress → color

#596b75

RGB (89, 107, 117) · 100.00% opacity

:root → --muted

#5b9d84

RGB (91, 157, 132) · 100.00% opacity

.timeline article:before → background

#5c7566

RGB (92, 117, 102) · 100.00% opacity

.campus-visit p → color

#62776a

RGB (98, 119, 106) · 100.00% opacity

.campus-facilities p → color

#63766f

RGB (99, 118, 111) · 100.00% opacity

.site-nav a → color

#647c6c

RGB (100, 124, 108) · 100.00% opacity

.campus-cover figcaption → color

#65a98d

RGB (101, 169, 141) · 100.00% opacity

.notification.unread → border-left

#668179

RGB (102, 129, 121) · 100.00% opacity

.welcome-banner p → color

#687b81

RGB (104, 123, 129) · 100.00% opacity

.sidebar nav a → color

#6a7b82

RGB (106, 123, 130) · 100.00% opacity

.top-actions a → color

#6b7e73

RGB (107, 126, 115) · 100.00% opacity

.carousel-heading → color; .carousel-position → color

#6b7e85

RGB (107, 126, 133) · 100.00% opacity

.stat-card>span → color

#6e7e86

RGB (110, 126, 134) · 100.00% opacity

:root → --muted

#708475

RGB (112, 132, 117) · 100.00% opacity

.campus-location-tag → color

#75858c

RGB (117, 133, 140) · 100.00% opacity

.announcement-preview p → color

#75858d

RGB (117, 133, 141) · 100.00% opacity

.empty → color

#76a88b

RGB (118, 168, 139) · 100.00% opacity

.campus-marker → background

#76d2bc

RGB (118, 210, 188) · 100.00% opacity

[data-theme=dark] → --teal

#784640

RGB (120, 70, 64) · 100.00% opacity

[data-theme=dark] :is(.alert.error, .errorlist) → border-color

#7b8a81

RGB (123, 138, 129) · 100.00% opacity

.campus-breadcrumb → color

#7c8a80

RGB (124, 138, 128) · 100.00% opacity

.campus-source → color

#8a9d91

RGB (138, 157, 145) · 100.00% opacity

.feature-number → color

#94a0a5

RGB (148, 160, 165) · 100.00% opacity

.workspace → color

#99a6aa

RGB (153, 166, 170) · 100.00% opacity

footer → color

#9a6b17

RGB (154, 107, 23) · 100.00% opacity

.pending → color

#a5b5be

RGB (165, 181, 190) · 100.00% opacity

[data-theme=dark] → --muted

#a5c0af

RGB (165, 192, 175) · 100.00% opacity

.footer-about>span → color

#a7523e

RGB (167, 82, 62) · 100.00% opacity

.alert.error → color

#a7c1b0

RGB (167, 193, 176) · 100.00% opacity

.footer-bottom → color

#a9c9b5

RGB (169, 201, 181) · 100.00% opacity

.site-footer .brand-dot → color

#ac4239

RGB (172, 66, 57) · 100.00% opacity

.errorlist → color

#b0e4cc

RGB (176, 228, 204) · 100.00% opacity

[data-theme=dark] .status → color

#b8efd0

RGB (184, 239, 208) · 100.00% opacity

[data-theme=dark] :is(.notification.unread, .alert) → color

#bd8c3a

RGB (189, 140, 58) · 100.00% opacity

.list-row:nth-child(3n) .file-icon → color

#c2d5c9

RGB (194, 213, 201) · 100.00% opacity

.footer-about p → color

#c4d8cb

RGB (196, 216, 203) · 100.00% opacity

.site-footer nav a → color

#c6d4ca

RGB (198, 212, 202) · 100.00% opacity

.carousel-dots button::after → background

#cadbd0

RGB (202, 219, 208) · 100.00% opacity

.header-sign-in → border

#cbe5d2

RGB (203, 229, 210) · 100.00% opacity

.alert → border

#cfe3d7

RGB (207, 227, 215) · 100.00% opacity

.art-square → box-shadow

#d3e0d9

RGB (211, 224, 217) · 100.00% opacity

.site-footer → color

#d5e4da

RGB (213, 228, 218) · 100.00% opacity

.footer-bottom a → color

#d6e1e3

RGB (214, 225, 227) · 100.00% opacity

input:not([type=checkbox]):not([type=radio]),select,textarea → border

#d7e7dc

RGB (215, 231, 220) · 100.00% opacity

.site-footer .brand-icon → background

#d8e2e4

RGB (216, 226, 228) · 100.00% opacity

.secondary → border-color

#d9e7dc

RGB (217, 231, 220) · 100.00% opacity

.campus-visit → border

#d9e9e0

RGB (217, 233, 224) · 100.00% opacity

.welcome-banner → border

#dbece4

RGB (219, 236, 228) · 100.00% opacity

.timeline → border-left

#dce8e0

RGB (220, 232, 224) · 100.00% opacity

.auth-tiles>div → border

#dceee5

RGB (220, 238, 229) · 100.00% opacity

.avatar → background

#dfe7e1

RGB (223, 231, 225) · 100.00% opacity

.campus-facilities article → border; .campus-photo-grid figure → border; .campus-space-card → border

#dfe8e2

RGB (223, 232, 226) · 100.00% opacity

.campus-location-tag → border-top; .campus-spaces → border-top; .home-features → border-top

#dfe9e1

RGB (223, 233, 225) · 100.00% opacity

.campus-photo-grid .campus-photo-portrait img → background

#dfece3

RGB (223, 236, 227) · 100.00% opacity

.campus-facilities .file-icon → background

#e0e8e2

RGB (224, 232, 226) · 100.00% opacity

.home-features .panel → border-color

#e0eae4

RGB (224, 234, 228) · 100.00% opacity

.campus-slide figcaption p → color

#e1ece5

RGB (225, 236, 229) · 100.00% opacity

.site-footer h2 → color

#e1f1e9

RGB (225, 241, 233) · 100.00% opacity

.badge → background

#e2e8e2

RGB (226, 232, 226) · 100.00% opacity

.campus-identity img → border

#e2e9e4

RGB (226, 233, 228) · 100.00% opacity

.site-header → border-bottom

#e3eee7

RGB (227, 238, 231) · 100.00% opacity

.campus-slide figcaption>span → color

#e5ebee

RGB (229, 235, 238) · 100.00% opacity

:root → --line

#e5eee7

RGB (229, 238, 231) · 100.00% opacity

.carousel-dots button:hover → background

#e5eeea

RGB (229, 238, 234) · 100.00% opacity

.tag → border

#e5efe7

RGB (229, 239, 231) · 100.00% opacity

.campus-marker → box-shadow

#e5f0e9

RGB (229, 240, 233) · 100.00% opacity

.auth-tiles>div → background

#e6edf1

RGB (230, 237, 241) · 100.00% opacity

[data-theme=dark] → --ink

#e6f0e8

RGB (230, 240, 232) · 100.00% opacity

.campus-visit → background

#e7af49

RGB (231, 175, 73) · 100.00% opacity

button:focus-visible,a:focus-visible,input:focus-visible,textarea:focus-visible,select:focus-visible → outline

#e7f2ec

RGB (231, 242, 236) · 100.00% opacity

.welcome-banner → background

#e7f4ec

RGB (231, 244, 236) · 100.00% opacity

.resolved → background

#e8f4eb

RGB (232, 244, 235) · 100.00% opacity

.alert → background

#e8f4ef

RGB (232, 244, 239) · 100.00% opacity

:root → --pale

#e9b55c

RGB (233, 181, 92) · 100.00% opacity

.art-circle → color

#eaf0ff

RGB (234, 240, 255) · 100.00% opacity

.in_progress → background

#edf3ee

RGB (237, 243, 238) · 100.00% opacity

.campus-facilities article → background

#edf3ef

RGB (237, 243, 239) · 100.00% opacity

progress → background; progress::-webkit-progress-bar → background

#eef5f1

RGB (238, 245, 241) · 100.00% opacity

.file-icon → background

#efd5ce

RGB (239, 213, 206) · 100.00% opacity

.alert.error → border-color

#f0f3f4

RGB (240, 243, 244) · 100.00% opacity

.announcement-preview → border-top; .list-row → border-top; td → border-bottom

#f0f5f3

RGB (240, 245, 243) · 100.00% opacity

.tag → background

#f0f7f3

RGB (240, 247, 243) · 100.00% opacity

.answer-box → background

#f0f8f3

RGB (240, 248, 243) · 100.00% opacity

.notification.unread → background

#f4d48d

RGB (244, 212, 141) · 100.00% opacity

[data-theme=dark] .status.pending → color

#f4f6f6

RGB (244, 246, 246) · 100.00% opacity

code → background

#f5f7fa

RGB (245, 247, 250) · 100.00% opacity

:root → --bg

#f7f9f7

RGB (247, 249, 247) · 100.00% opacity

.public-site → background

#fafcfb

RGB (250, 252, 251) · 100.00% opacity

.status-grid>div → background

#ffc2ba

RGB (255, 194, 186) · 100.00% opacity

[data-theme=dark] :is(.alert.error, .errorlist) → color

#fff

RGB (255, 255, 255) · 100.00% opacity

.campus-cover → background; .campus-cover → border; .campus-identity img → background; .campus-photo-grid figure → background; .campus-slide figcaption → color; .campus-space-card → background; .carousel-arrow:hover → border-color; .home-features .panel → background; .light → background; .light → border-color; .role-switch a[aria-current=page] → color; .site-footer .brand → color; .site-footer a:hover → color; .skip-link → color; :root → --white; [data-theme=dark] :is(button, .button):not(.secondary):not(.theme-toggle) → color;
input:not([type=checkbox]):not([type=radio]),select,textarea → background

#fff0ed

RGB (255, 240, 237) · 100.00% opacity

.errorlist → background

#fff1ed

RGB (255, 241, 237) · 100.00% opacity

.alert.error → background

#fff3dc

RGB (255, 243, 220) · 100.00% opacity

.pending → background

#fff5e6

RGB (255, 245, 230) · 100.00% opacity

.list-row:nth-child(3n) .file-icon → background

#ffffff20

RGB (255, 255, 255) · 12.55% opacity

.footer-bottom → border-top

#ffffff80

RGB (255, 255, 255) · 50.20% opacity

.carousel-arrow → border

#ffffffc9

RGB (255, 255, 255) · 78.82% opacity

.topbar → background

#fffffff2

RGB (255, 255, 255) · 94.90% opacity

.carousel-arrow → background

#fffffff5

RGB (255, 255, 255) · 96.08% opacity

.site-header → background

transparent

RGB (0, 0, 0) · 0.00% opacity

.campus-slide figcaption → background; .carousel-dots button → background

white

RGB (255, 255, 255) · 100.00% opacity

.art-small → color; .art-square → background; .brand-icon → color; .panel → background; .sidebar → background; .stat-card → background; .timeline article:before → border; button,.button → color; button:hover,.button:hover → color